

VCC: Assignment 3

Name: Ashutosh Nigam, Roll No: M25AI2006

Objective:

Create a local VM and implement a mechanism to monitor resource usage. Configure it to auto-scale to a public cloud (e.g., GCP, AWS, or Azure) when resource usage exceeds 75%.

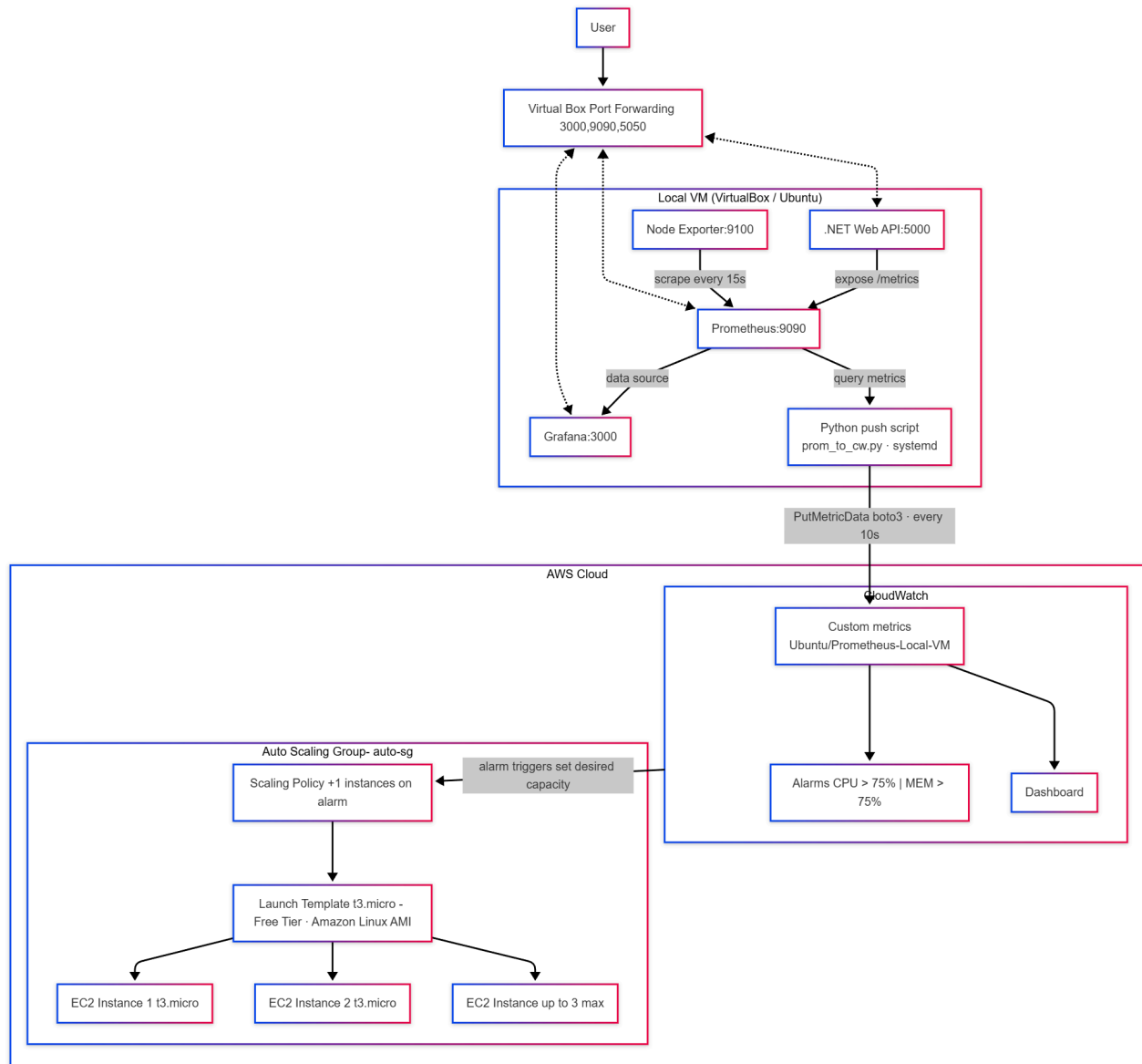
Deliverables:

1. Document Report:
 - Step-by-Step Instructions for Implementation
 - Creation of a local VM (using VirtualBox, VMware, or similar).
 - Implementation of resource monitoring (using tools like Prometheus, Grafana, or a custom script).
 - Configuration of cloud auto-scaling policies and resource migration.
 - Deployment of a sample application to demonstrate the process.
 2. Architecture Design:
 - Diagram illustrating the flow: local VM resource monitoring → scaling to cloud → deployment.
 3. Link to Source Code Repo:
 - Repository containing monitoring scripts, deployment configurations, and application code.
 4. Link to Recorded Video Demo:
 - A video showing the setup, resource monitoring, and auto-scaling process in action, with a detailed voice-over.
-

Repository Link:  GitHub - mrashutoshnigam/VCC.Assignment3

<https://github.com/mrashutoshnigam/VCC.Assignment3>

Architecture Diagram:

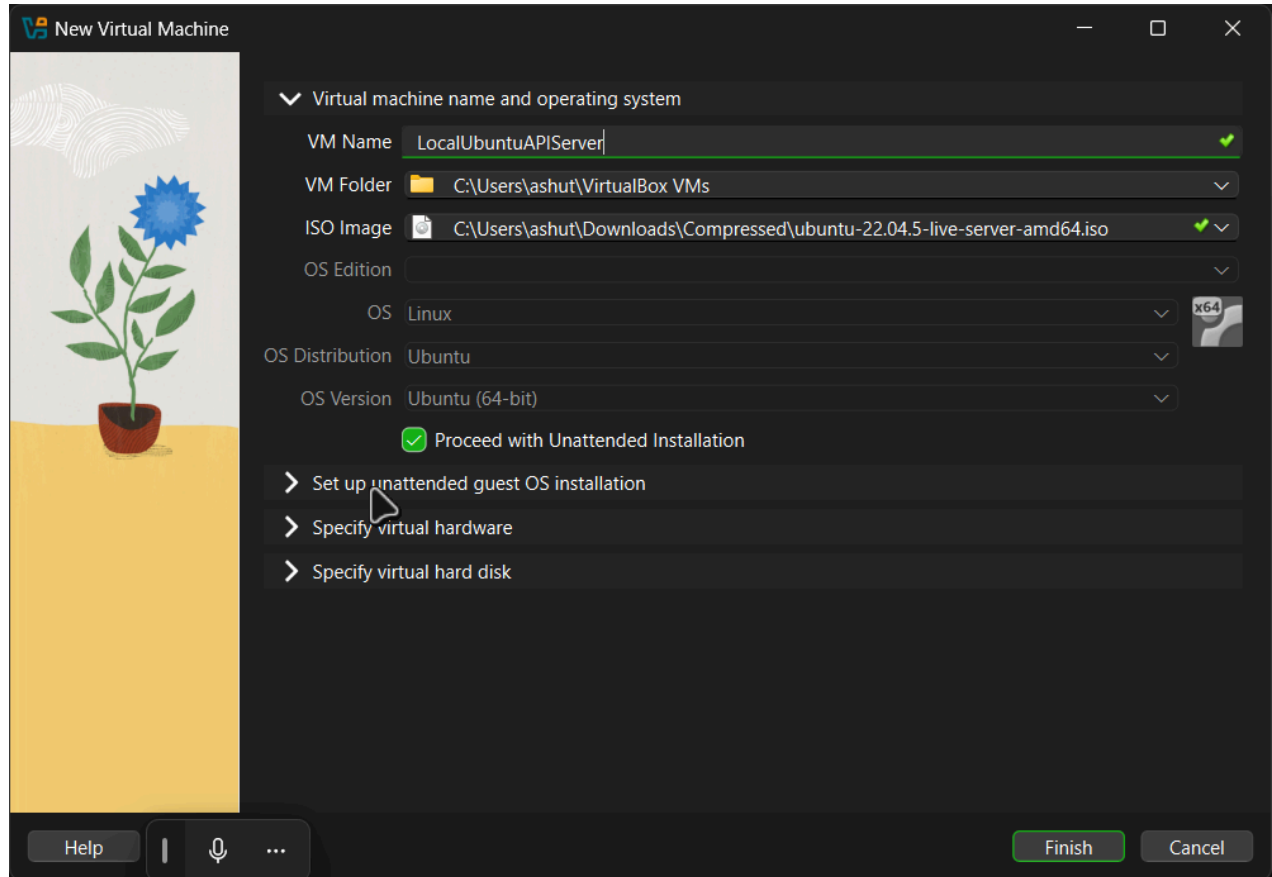


Implementation Steps:

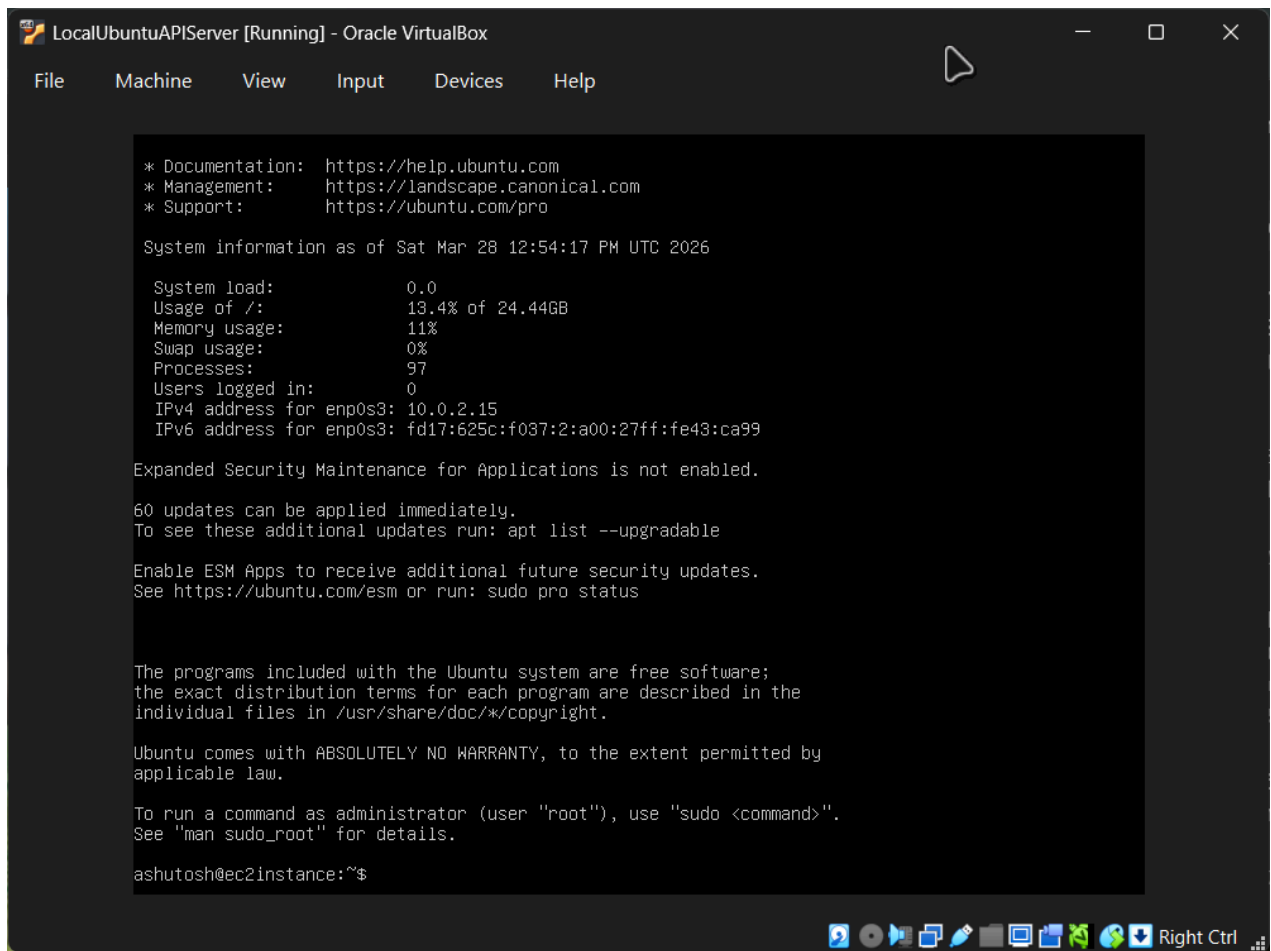
1. I created a Microsoft ASP.NET Core Web API application that serves as a **System Resource Monitor and Load Generator** designed to test and demonstrate hybrid cloud auto-scaling capabilities. The application provides endpoints to:
 - a. Monitor system resources (CPU, Memory)
 - b. Trigger controlled CPU spikes
 - c. Trigger controlled Memory spikes
 - d. Provide real-time system status
2. Created a virtual machine in Virtual Box using Ubuntu-Server Image
3. Installed .NET and Configured WebAPI to run on this Server.
4. Installed Prometheus and Grafana and Node Exporter in the server to see metrics
5. Installed AWS Client and configured it with my AWS personal account access keys and secrets
6. Created a Script to push metrics to Cloud watch
7. Created Cloud watch alerts based on CPU and Memory Uses
8. Configured A launch Template using Amazon Linux Image `al2023-ami-2023.10.20260325.0-kernel-6.1-x86_64` and `t3.micro` configuration
9. Created a Auto Scaling Group named `auto-sg` to launch templates

10. Configured auto scaling group to listen to matrices and alters from Cloud watch alarms
11. I tested it.

Virtual Machine Setup

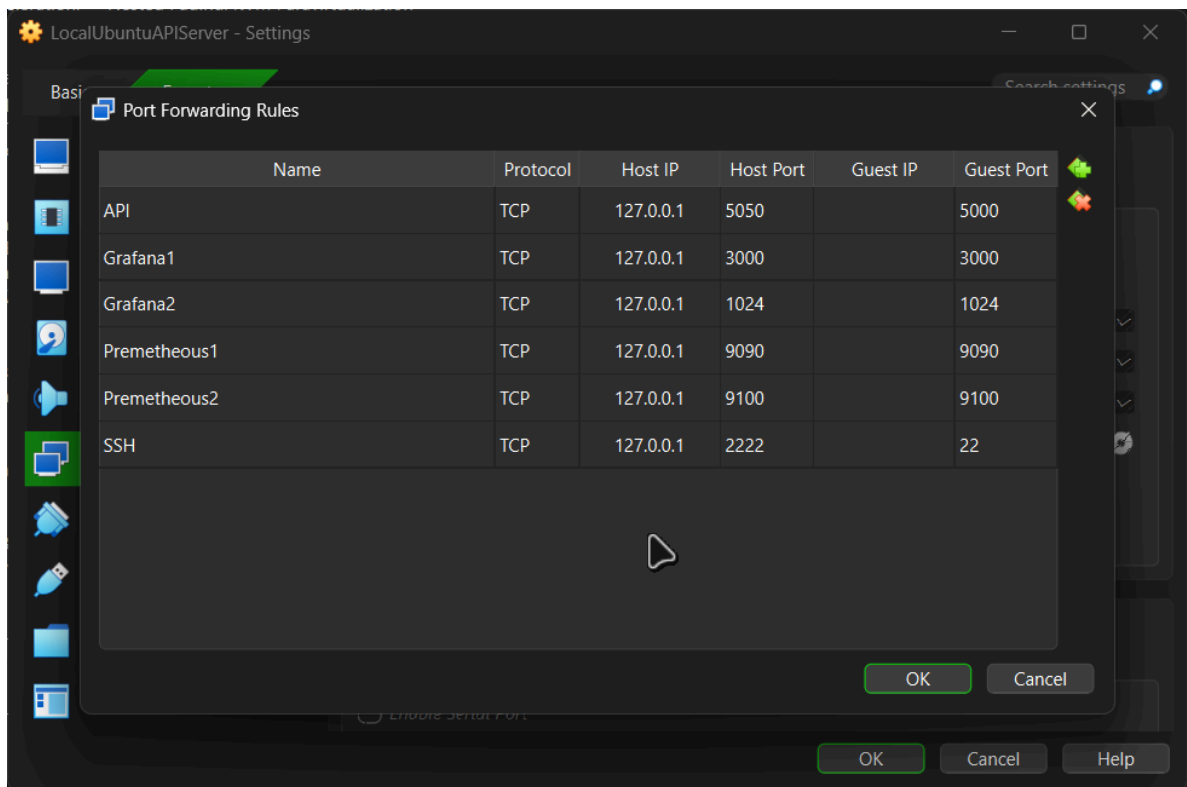


2.



3. Port Forwarding Setup

a.



b. Run following to update

- i. `sudo apt update & sudo apt upgrade -y`
- c. Install `openssh` for SSH `sudo apt install openssh-server -y`
- d.

```

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Sat Mar 28 01:48:05 PM UTC 2026

System load:            0.0
Usage of /:              13.5% of 24.44GB
Memory usage:           11%
Swap usage:             0%
Processes:              102
Users logged in:        1
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe43:ca99

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Mar 28 13:48:05 2026 from 10.0.2.2
ashutosh@ec2instance:~$

```

- e. Install Microsoft.NET from [Install .NET on Ubuntu - .NET | Microsoft Le...](#)
 - i. `sudo apt-get update && sudo apt-get install -y dotnet-sdk-8.0`
 - ii. `dotnet --info`
- f.

```

Last login: Sat Mar 28 13:48:05 2026 from 10.0.2.2
ashutosh@ec2instance:~$ sudo apt-get update & sudo apt-get install -y dotnet-sdk-8.0
[1] 1052
[sudo] password for ashutosh:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  aspnetcore-runtime-8.0 aspnetcore-targeting-pack-8.0 dotnet-apphost-pack-8.0 dotnet-host-8.0 dotnet-hostfxr-8.0
  dotnet-runtime-8.0 dotnet-targeting-pack-8.0 dotnet-templates-8.0 liblttng-ust-common1 liblttng-ust-ctl5
  liblttng-ust1 netstandard-targeting-pack-2.1-8.0
Suggested packages:
  aspnetcore-runtime-dbg-8.0 dotnet-runtime-dbg-8.0 dotnet-sdk-dbg-8.0
The following NEW packages will be installed:
  aspnetcore-runtime-8.0 aspnetcore-targeting-pack-8.0 dotnet-apphost-pack-8.0 dotnet-host-8.0 dotnet-hostfxr-8.0
  dotnet-runtime-8.0 dotnet-sdk-8.0 dotnet-targeting-pack-8.0 dotnet-templates-8.0 liblttng-ust-common1
  liblttng-ust-ctl5 liblttng-ust1 netstandard-targeting-pack-2.1-8.0
0 upgraded, 13 newly installed, 0 to remove and 0 not upgraded.
Need to get 138 MB of archives.
After this operation, 495 MB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 dotnet-host-8.0 amd64 8.0.25-0ubuntu1~22.04.1 [18
8 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 dotnet-hostfxr-8.0 amd64 8.0.25-0ubuntu1~22.04.1
[147 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu jammy/main amd64 liblttng-ust-common1 amd64 2.13.1-1ubuntu1 [27.1 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu jammy/main amd64 liblttng-ust-ctl5 amd64 2.13.1-1ubuntu1 [77.9 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu jammy/main amd64 liblttng-ust1 amd64 2.13.1-1ubuntu1 [190 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 dotnet-runtime-8.0 amd64 8.0.25-0ubuntu1~22.04.1

```

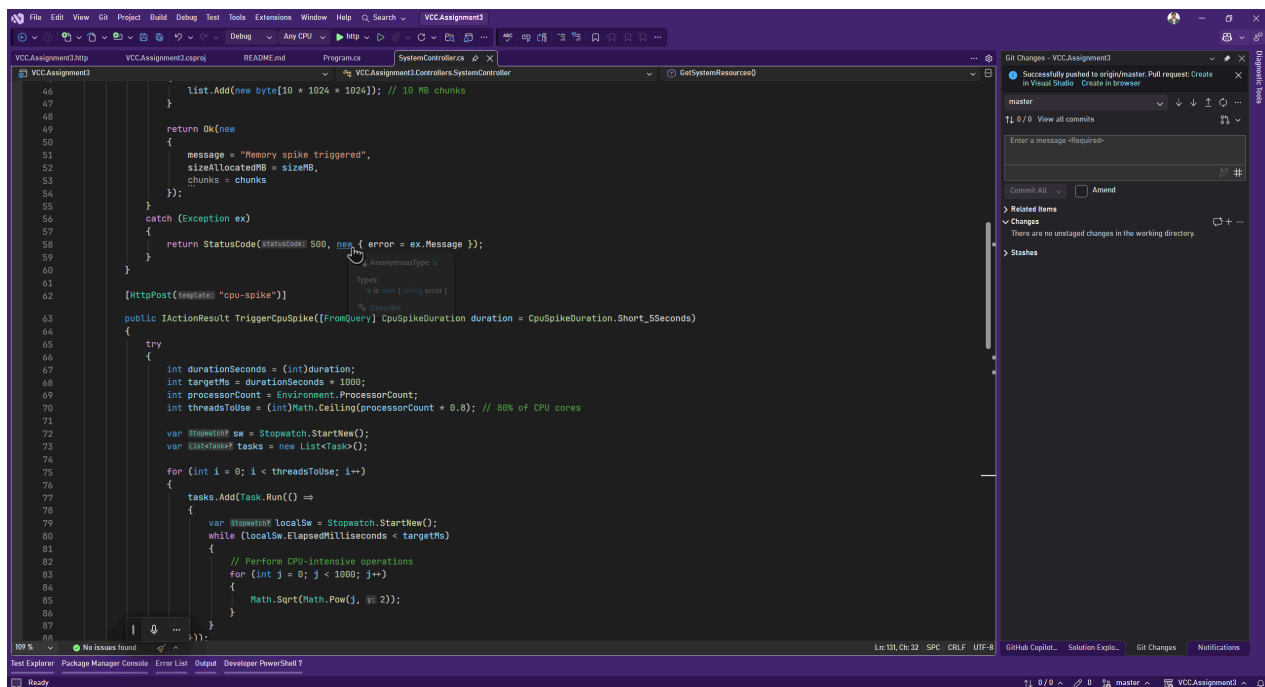
- Run command to add it to firewall
- `sudo ufw allow 5000`
- `sudo ufw allow 22`

- Configure API as service
- `sudo mkdir /var/www`
- `sudo chown ashutosh:ashutosh /var/www/`
- `nano /etc/systemd/system/api.service`

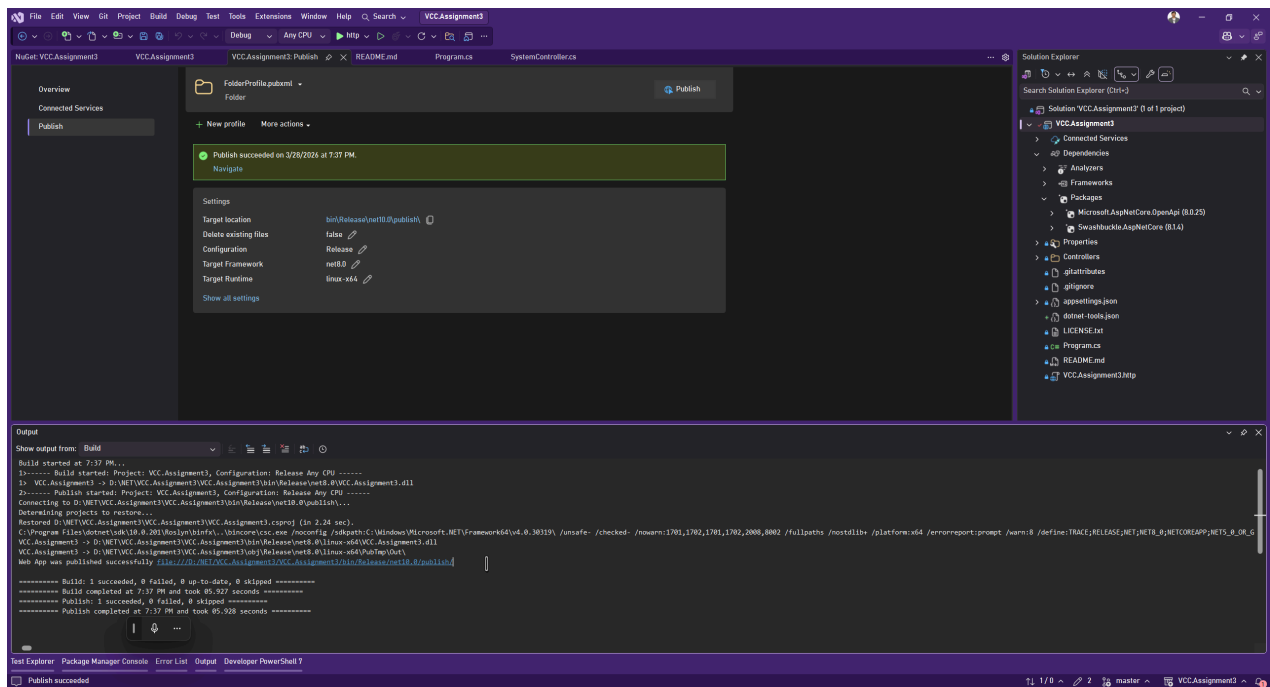
</> Shell

```
1 [Unit]
2 Description=API Service
3
4 [Service]
5 WorkingDirectory=/var/www
6 ExecStart=dotnet VCC.Assignment3.dll --urls http://0.0.0.0:5000
7 Restart=always
8 RestartSec=5
9
10 [Install]
11 WantedBy=multi-user.target
```

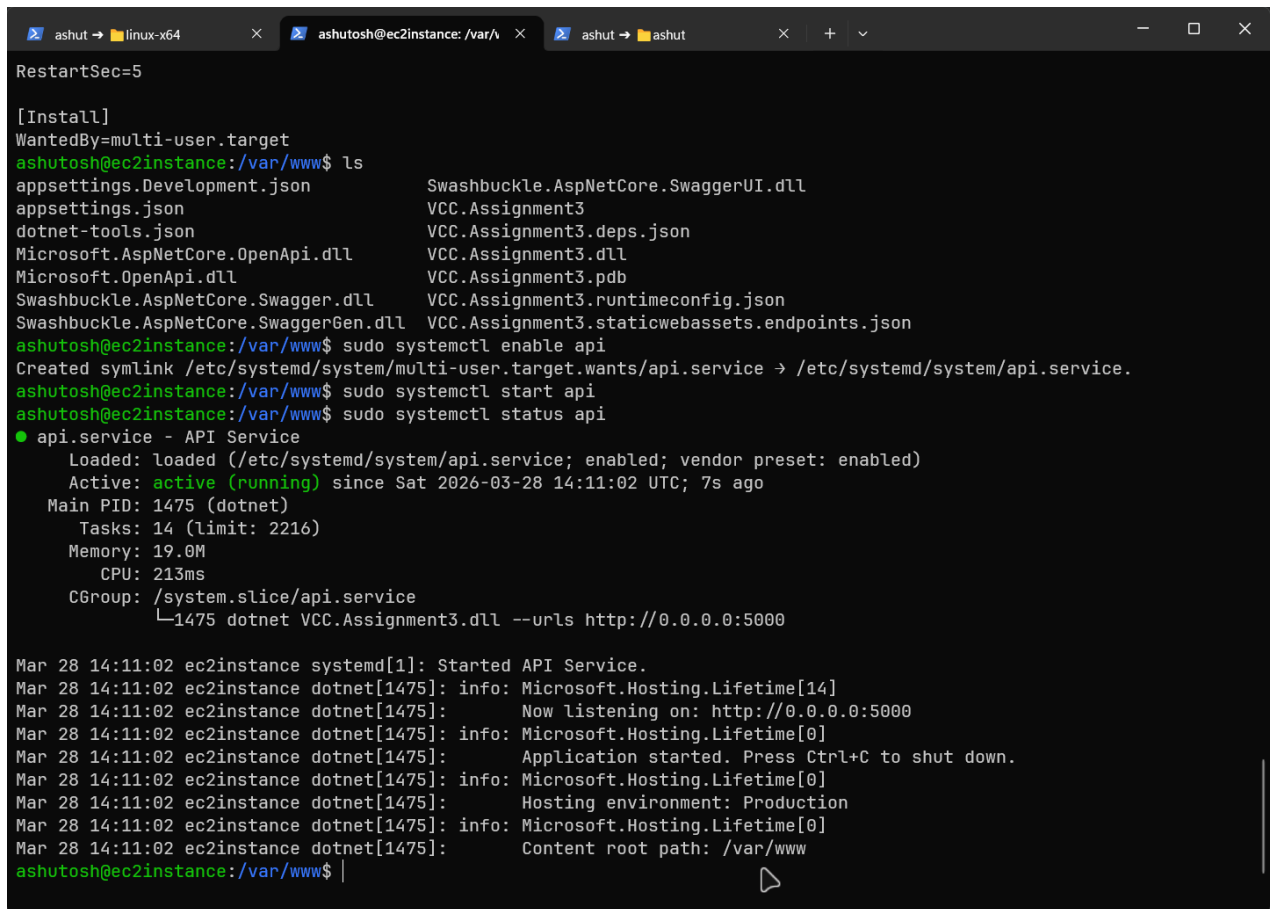
Created a .NET Web API based application



- Published .NET Project



- Copied Code to VM Server
- `scp -r -P 2222 .\ ashutosh@127.0.0.1:/var/www`
- `sudo systemctl enable api`
- `sudo systemctl start api`



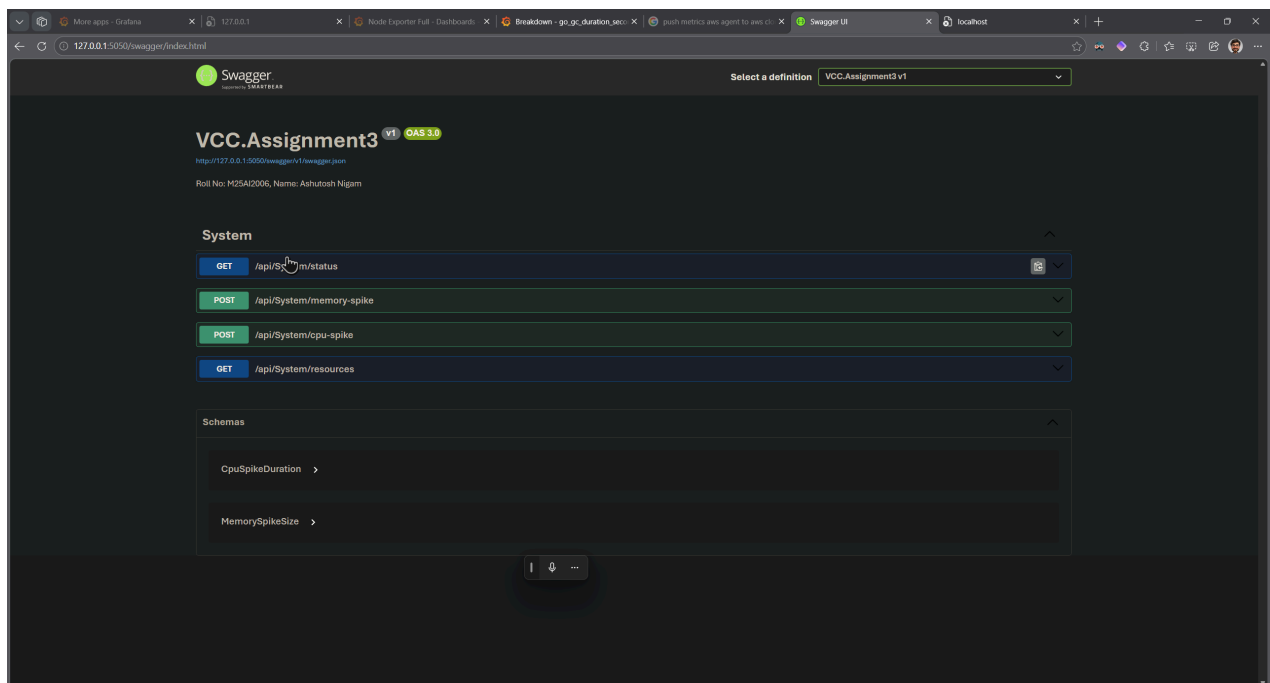
- API is running

```

ashutosh@ec2instance: /var/www$ ls
appsettings.Development.json  Swashbuckle.AspNetCore.SwaggerUI.dll
appsettings.json              VCC.Assignment3
dotnet-tools.json             VCC.Assignment3.deps.json
Microsoft.AspNetCore.OpenApi.dll  VCC.Assignment3.dll
Microsoft.OpenApi.dll         VCC.Assignment3.pdb
Swashbuckle.AspNetCore.Swagger.dll  VCC.Assignment3.runtimeconfig.json
Swashbuckle.AspNetCore.SwaggerGen.dll  VCC.Assignment3.staticwebassets.endpoints.json
ashutosh@ec2instance: /var/www$ sudo systemctl enable api
Created symlink /etc/systemd/system/multi-user.target.wants/api.service → /etc/systemd/system/api.service.
ashutosh@ec2instance: /var/www$ sudo systemctl start api
ashutosh@ec2instance: /var/www$ sudo systemctl status api
● api.service - API Service
   Loaded: loaded (/etc/systemd/system/api.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-03-28 14:11:02 UTC; 7s ago
     Main PID: 1475 (dotnet)
       Tasks: 14 (limit: 2216)
      Memory: 19.0M
         CPU: 213ms
    CGroup: /system.slice/api.service
            └─1475 dotnet VCC.Assignment3.dll --urls http://0.0.0.0:5000

Mar 28 14:11:02 ec2instance systemd[1]: Started API Service.
Mar 28 14:11:02 ec2instance dotnet[1475]: info: Microsoft.Hosting.Lifetime[14]
Mar 28 14:11:02 ec2instance dotnet[1475]:   Now listening on: http://0.0.0.0:5000
Mar 28 14:11:02 ec2instance dotnet[1475]: info: Microsoft.Hosting.Lifetime[0]
Mar 28 14:11:02 ec2instance dotnet[1475]:   Application started. Press Ctrl+C to shut down.
Mar 28 14:11:02 ec2instance dotnet[1475]: info: Microsoft.Hosting.Lifetime[0]
Mar 28 14:11:02 ec2instance dotnet[1475]:   Hosting environment: Production
Mar 28 14:11:02 ec2instance dotnet[1475]: info: Microsoft.Hosting.Lifetime[0]
Mar 28 14:11:02 ec2instance dotnet[1475]:   Content root path: /var/www
ashutosh@ec2instance: /var/www$ curl http://0.0.0.0:5000/api/system/status
{"status": "API is running"}ashutosh@ec2instance: /var/www$

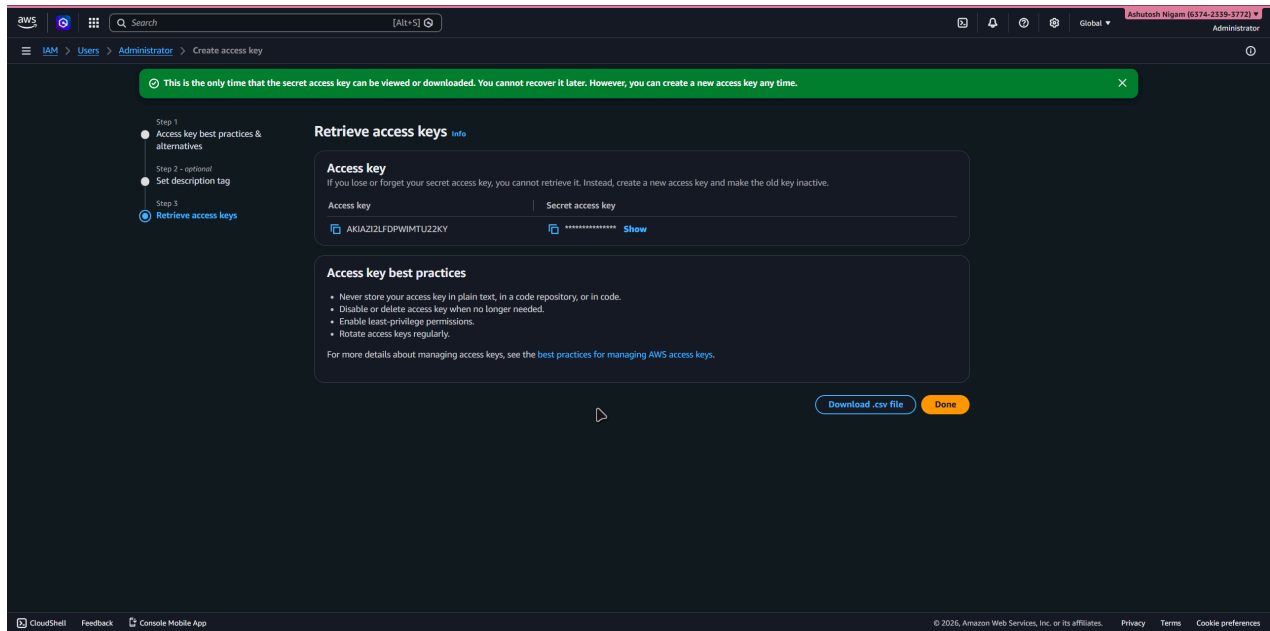
```



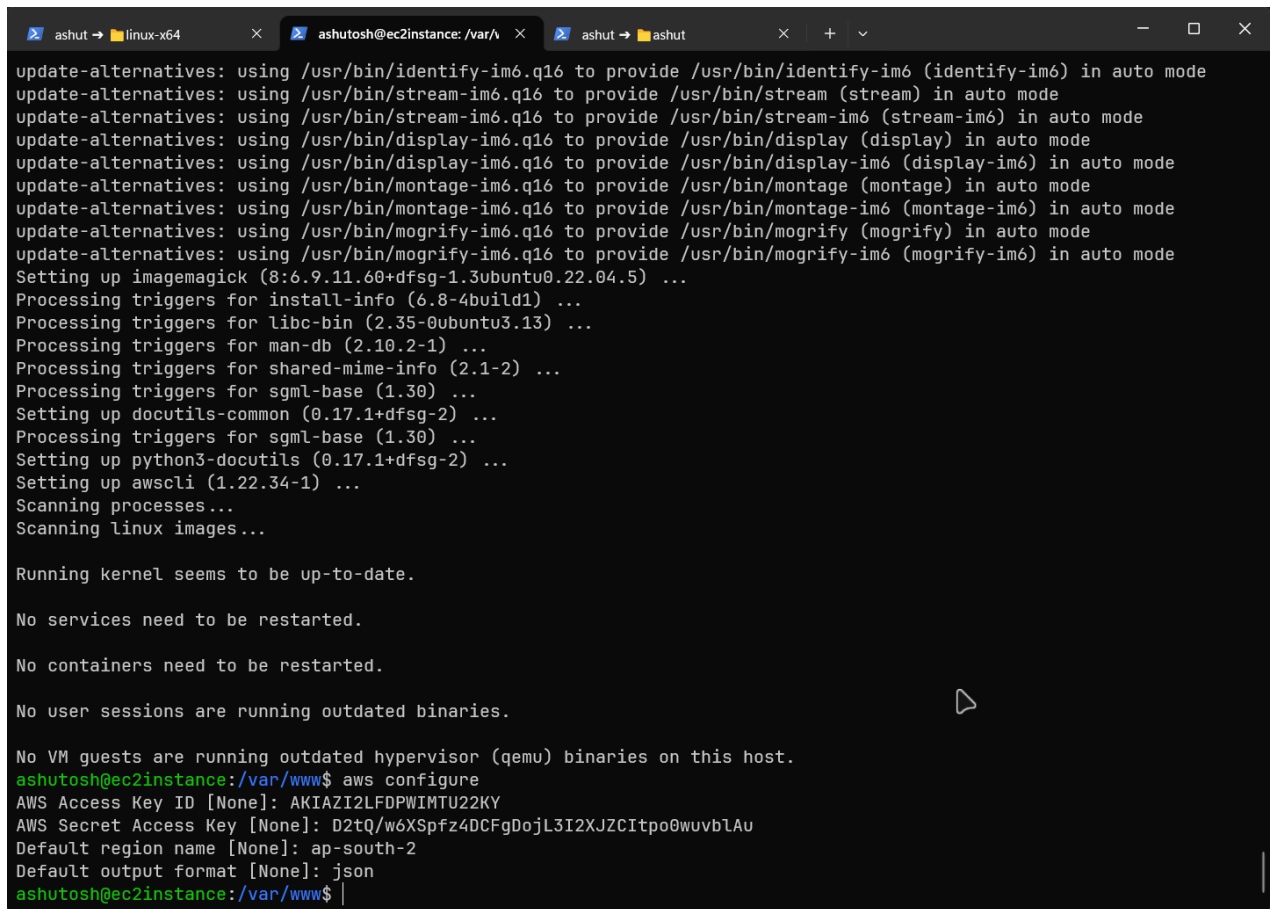
Configuring AWS CLI in VM Machine

- `sudo apt install awscli`

- `aws configure`
- Get API Keys from AWS



- Configure AWS Api Keys.



Configure Prometheus, Grafana, and Node Exporter

Steps:  Set up Grafana monitoring | Grafana docum...

How to Install and Configure Prometheus and Grafana on U...

How to Install and Configure Prometheus and Grafana on Ubuntu

<https://www.linode.com/docs/guides/how-to-install-prometheus-and-grafana-on-ubuntu/>

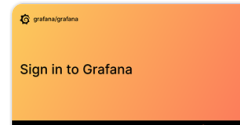


Installing Grafana

Sign in to Grafana | Grafana documentation

Learn how to sign in to Grafana

<https://grafana.com/docs/grafana/latest/setup-grafana/sign-in-to-grafana/>



- Install Prometheus
- `wget https://github.com/prometheus/prometheus/releases/download/v2.37.6/prometheus-2.37.6.linux-amd64.tar.gz`
- `tar xvfz prometheus-*.tar.gz`
- `rm prometheus-*.tar.gz`
- `sudo mkdir /etc/prometheus /var/lib/prometheus`
- `cd prometheus-2.37.6.linux-amd64`
- `sudo mv prometheus promtool /usr/local/bin/`
- `sudo mv prometheus.yml /etc/Prometheus/prometheus.yml`
- `sudo mv consoles/ console_libraries/ /etc/prometheus/`
- `prometheus --version`
-

```
ashutosh@ec2instance: ~$ ssh -p 2222 ashutosh@127.0.0.1
ashutosh@127.0.0.1's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-173-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Mar 29 06:46:43 AM UTC 2026

System load:  0.23
Usage of /:   24.8% of 24.44GB
Memory usage: 23%
Swap usage:   0%
Processes:   109
Users logged in: 1
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe43:ca99

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.
   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

9 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sun Mar 29 06:46:43 2026 from 10.0.2.2
ashutosh@ec2instance:~$ prometheus --version
prometheus, version 2.37.6 (branch: HEAD, revision: 8ade24a23af6be0f35414d6e8ce09598446c29a2)
 build user:   root@5f96027a7c3e
 build date:   20230220-09:36:40
 go version:   go1.19.6
 platform:    linux/amd64
ashutosh@ec2instance:~$
```

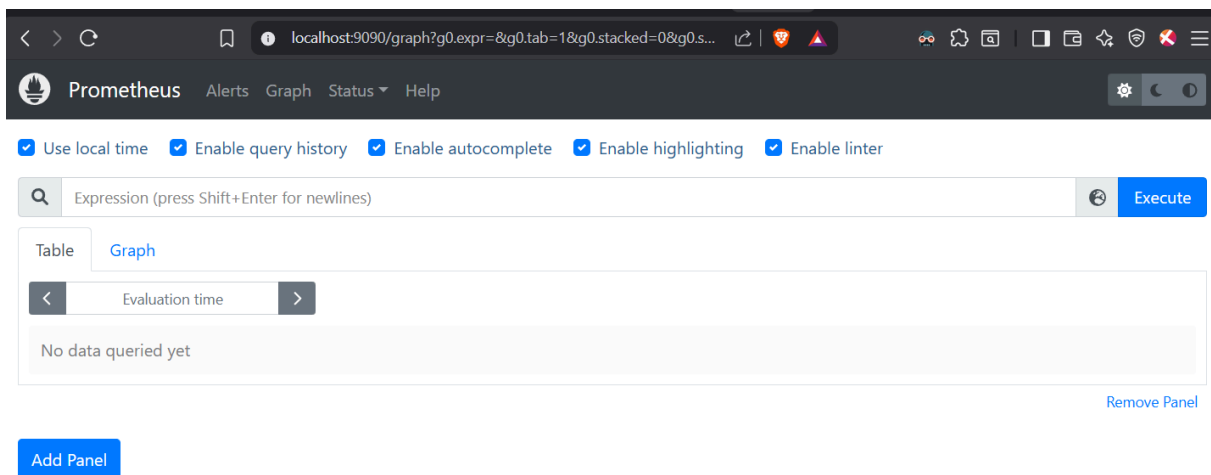
- Configure Prometheus as Service
- `sudo useradd -rs /bin/false prometheus`
- `sudo chown -R prometheus: /etc/prometheus /var/lib/prometheus`
- `sudo vi /etc/systemd/system/prometheus.service`

```

1 [Unit]
2 Description=Prometheus
3 Wants=network-online.target
4 After=network-online.target
5
6 [Service]
7 User=prometheus
8 Group=prometheus
9 Type=simple
10 Restart=on-failure
11 RestartSec=5s
12 ExecStart=/usr/local/bin/prometheus \
13     --config.file /etc/prometheus/prometheus.yml \
14     --storage.tsdb.path /var/lib/prometheus/ \
15     --web.console.templates=/etc/prometheus/consoles \
16     --web.console.libraries=/etc/prometheus/console_libraries \
17     --web.listen-address=0.0.0.0:9090 \
18     --web.enable-lifecycle \
19     --log.level=info
20
21 [Install]
22 WantedBy=multi-user.target

```

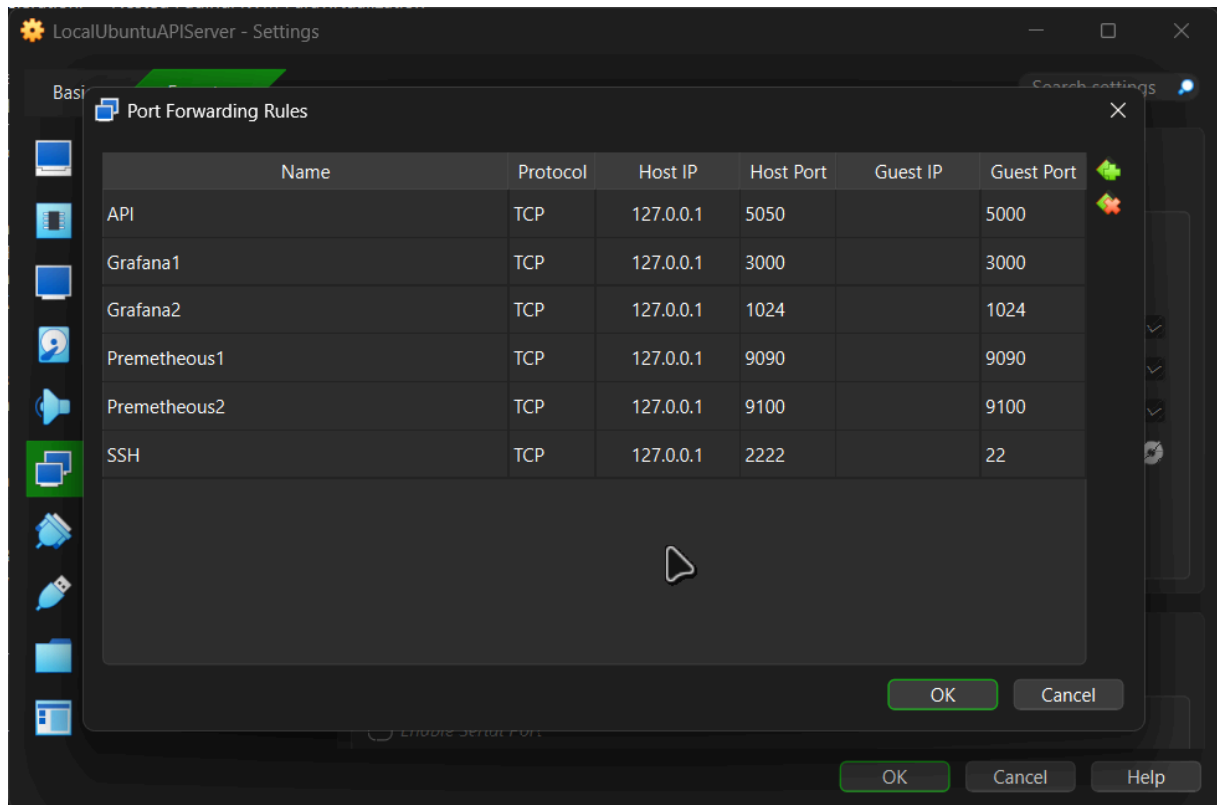
- `sudo systemctl daemon-reload`
- `sudo systemctl enable prometheus`
- `sudo systemctl start prometheus sudo systemctl status prometheus`

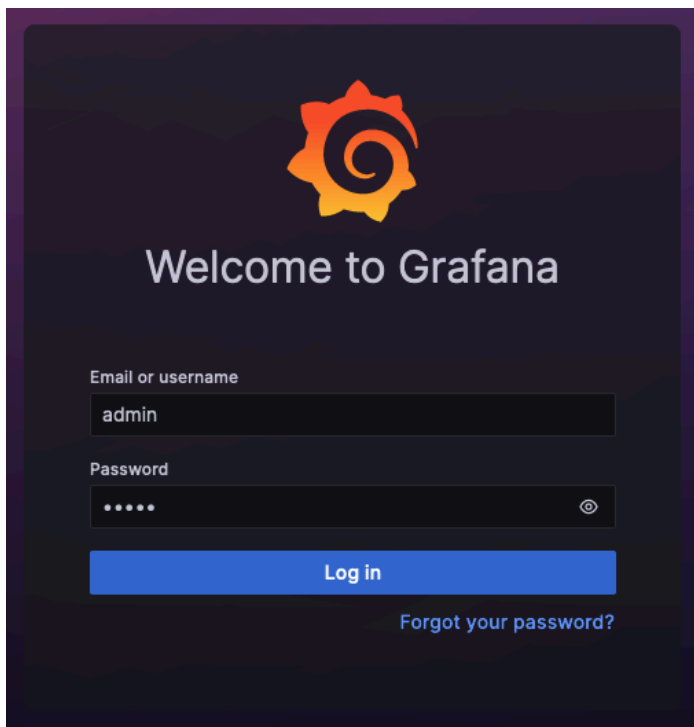


Configure Grafana

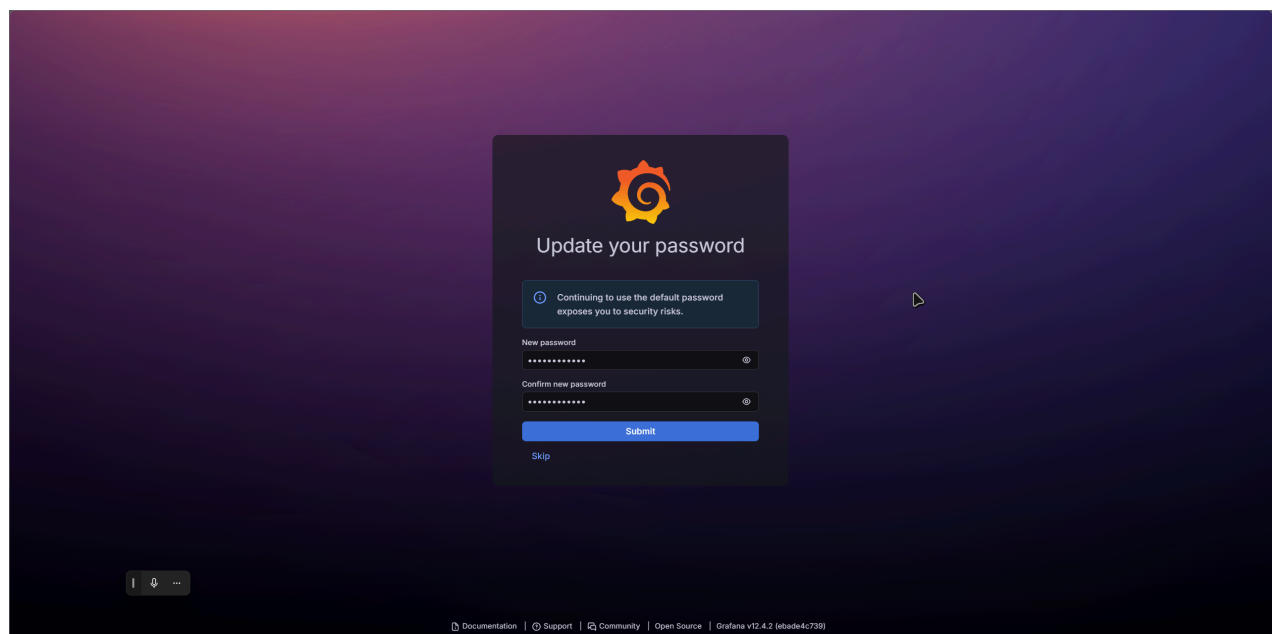
- `sudo apt-get install -y apt-transport-https wget gnupg`
- `sudo mkdir -p /etc/apt/keyrings`
- `sudo wget -O /etc/apt/keyrings/grafana.asc https://apt.grafana.com/gpg-full.key`
- `sudo chmod 644 /etc/apt/keyrings/grafana.asc`
- `echo "deb [signed-by=/etc/apt/keyrings/grafana.asc] https://apt.grafana.com stable main" | sudo`

- `tee -a /etc/apt/sources.list.d/grafana.list`
- `echo "deb [signed-by=/etc/apt/keyrings/grafana.asc] https://apt.grafana.com beta main" | sudo tee -a /etc/apt/sources.list.d/grafana.list`
- `sudo apt-get update`
- `sudo apt-get install grafana`
- Configure it as service
- `sudo systemctl daemon-reload`
- `sudo systemctl enable grafana-server.service`
- `sudo systemctl start Grafana-server`
- Added Ports in Virtual Machine
-





-



Configure Node Exporter:

- `wget https://github.com/prometheus/node\_exporter/releases/download/v1.5.0/node\_exporter-1.5.0.linux-amd64.tar.gz`
- `tar xvfz node_exporter-*.tar.gz`
- `tar xvfz node_exporter-*.tar.gz`
- `sudo mv node_exporter-1.5.0.linux-amd64/node_exporter /usr/local/bin`
- `rm -r node_exporter-1.5.0.linux-amd64*`
- `node_exporter`
- `sudo useradd -rs /bin/false node_exporter`

- Add it as service

- `sudo vi /etc/systemd/system/node_exporter.service`

-

```

1 [Unit]
2 Description=Node Exporter
3 Wants=network-online.target
4 After=network-online.target
5
6 [Service]
7 User=node_exporter
8 Group=node_exporter
9 Type=simple
10 Restart=on-failure
11 RestartSec=5s
12 ExecStart=/usr/local/bin/node_exporter
13
14 [Install]
15 WantedBy=multi-user.target

```

- `sudo systemctl enable node_exporter`
- `sudo systemctl daemon-reload``
- `sudo systemctl start node_exporter`
- `sudo systemctl status node_exporter`

Configure Prometheus to Monitor Client Node:

- `sudo vi /etc/prometheus/prometheus.yml`

```

GNU nano 6.2 /etc/prometheus/prometheus.yml
# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"
remote_writes:
  - url: https://aps-workspaces.us-east-1.amazonaws.com/workspaces/637423393772/api/v1/remote_write
    sigv4:
      region: ap-south-2
      access_key:
      secret_key:
    queue_config:
      max_samples_per_send: 1000
      max_shards: 200
      capacity: 2500

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label 'job=<job_name>' to any timeseries scraped from this config.
  - job_name: "prometheus"
    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.
    scrape_interval: 10s
    static_configs:
      - targets: ["localhost:9090"]

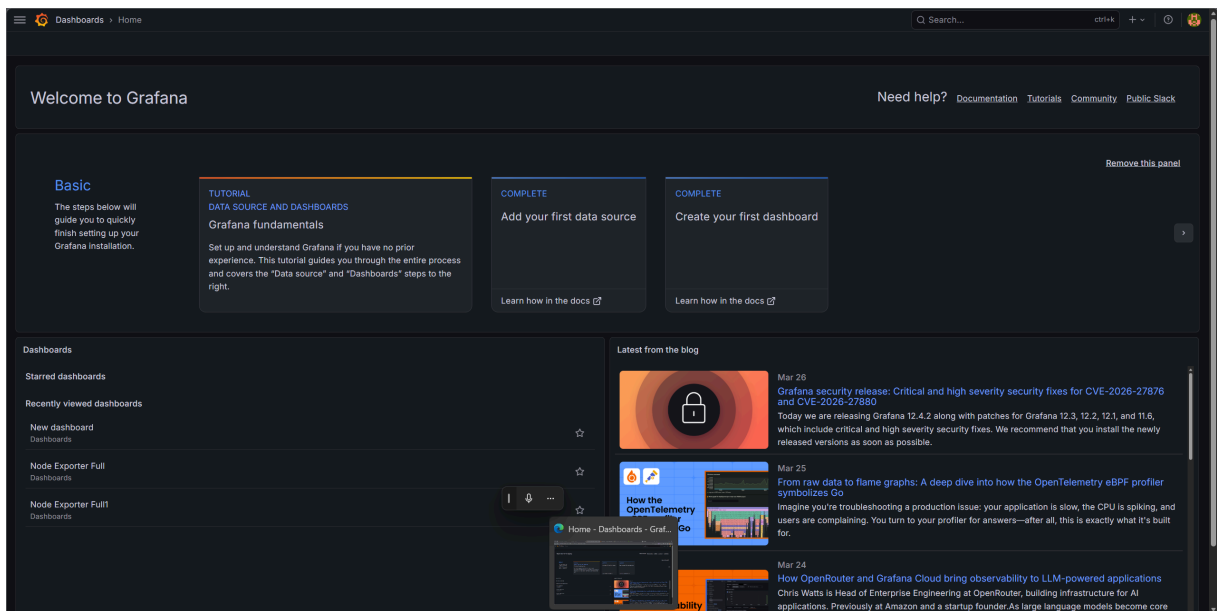
  - job_name: node
    static_configs:
      - targets: ["localhost:9100"]

```

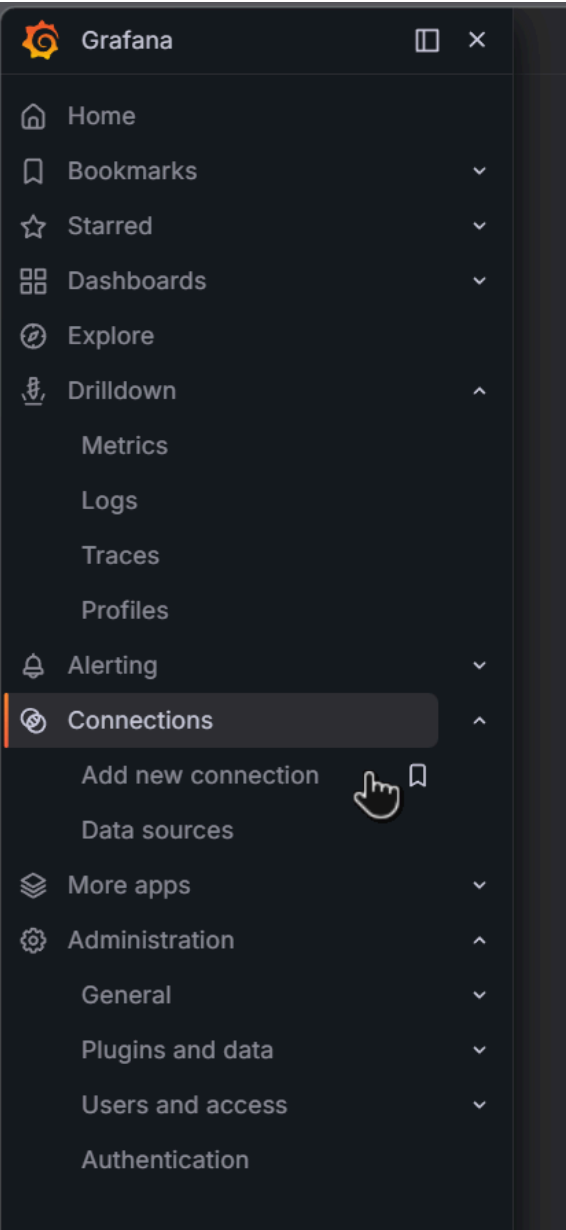
- `sudo systemctl restart prometheus`

Configure Grafana and Prometheus

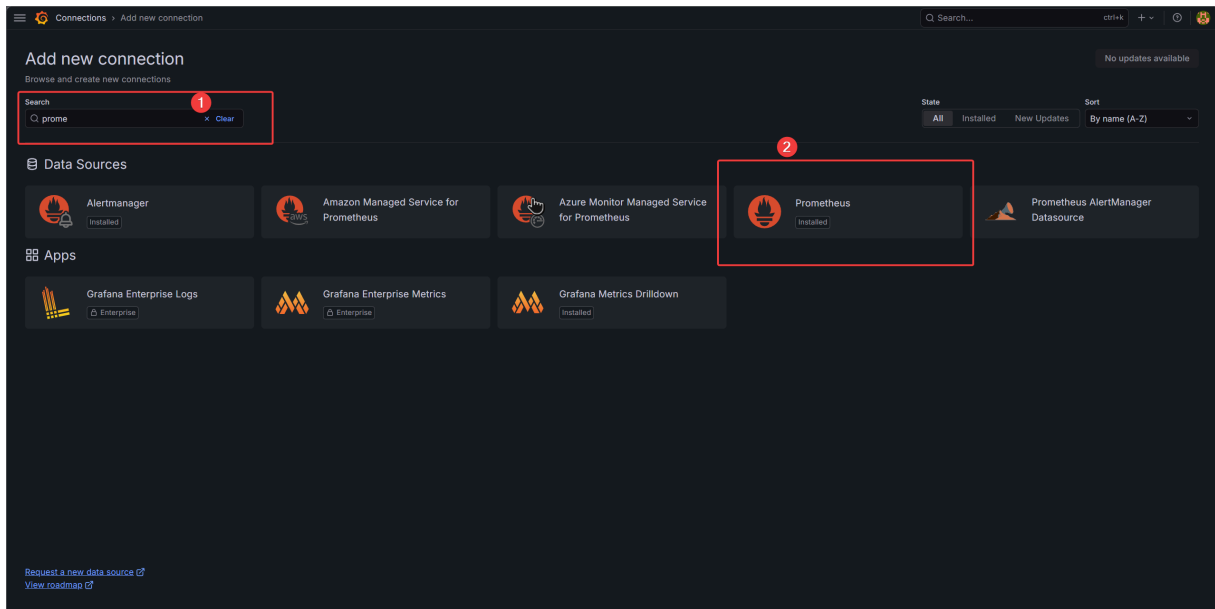
- Login to Grafana
-



- Select 'Add New Connection'
-

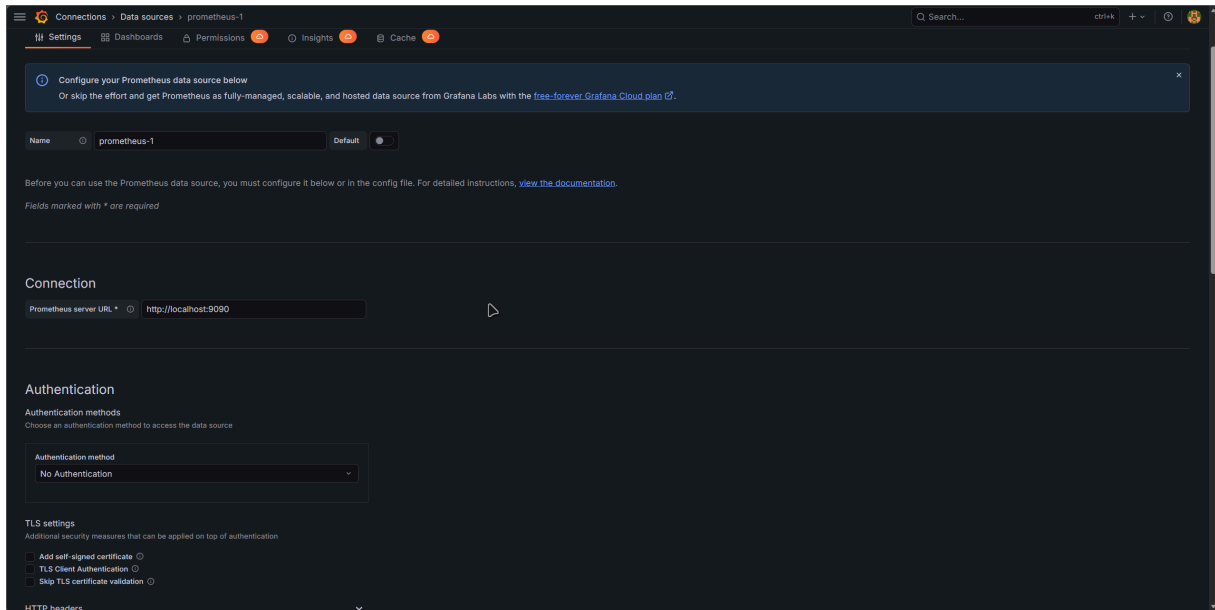


- Search Prometheus and select
-

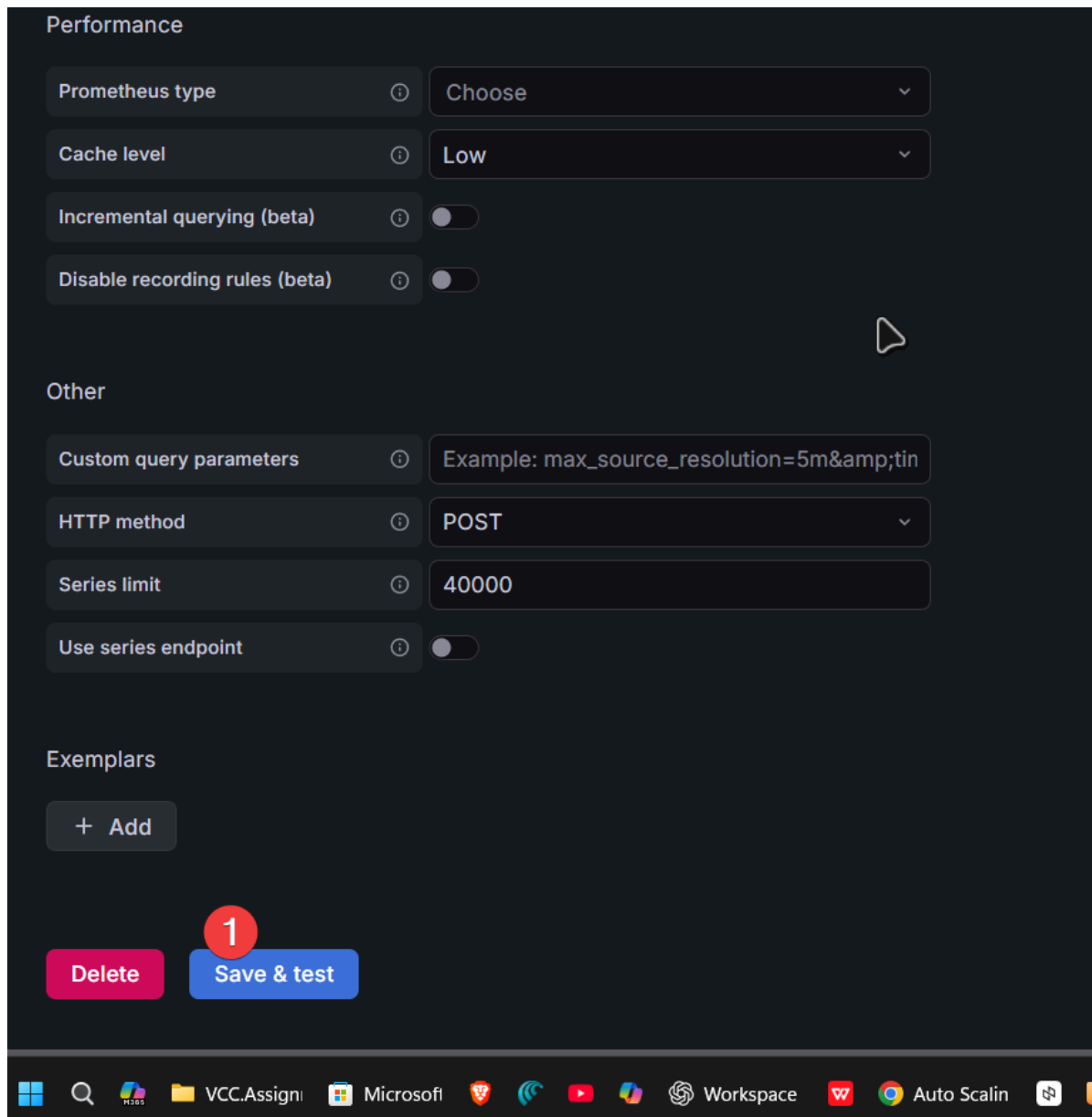


- Install and configure

•



- Save:



Configure Dashboard

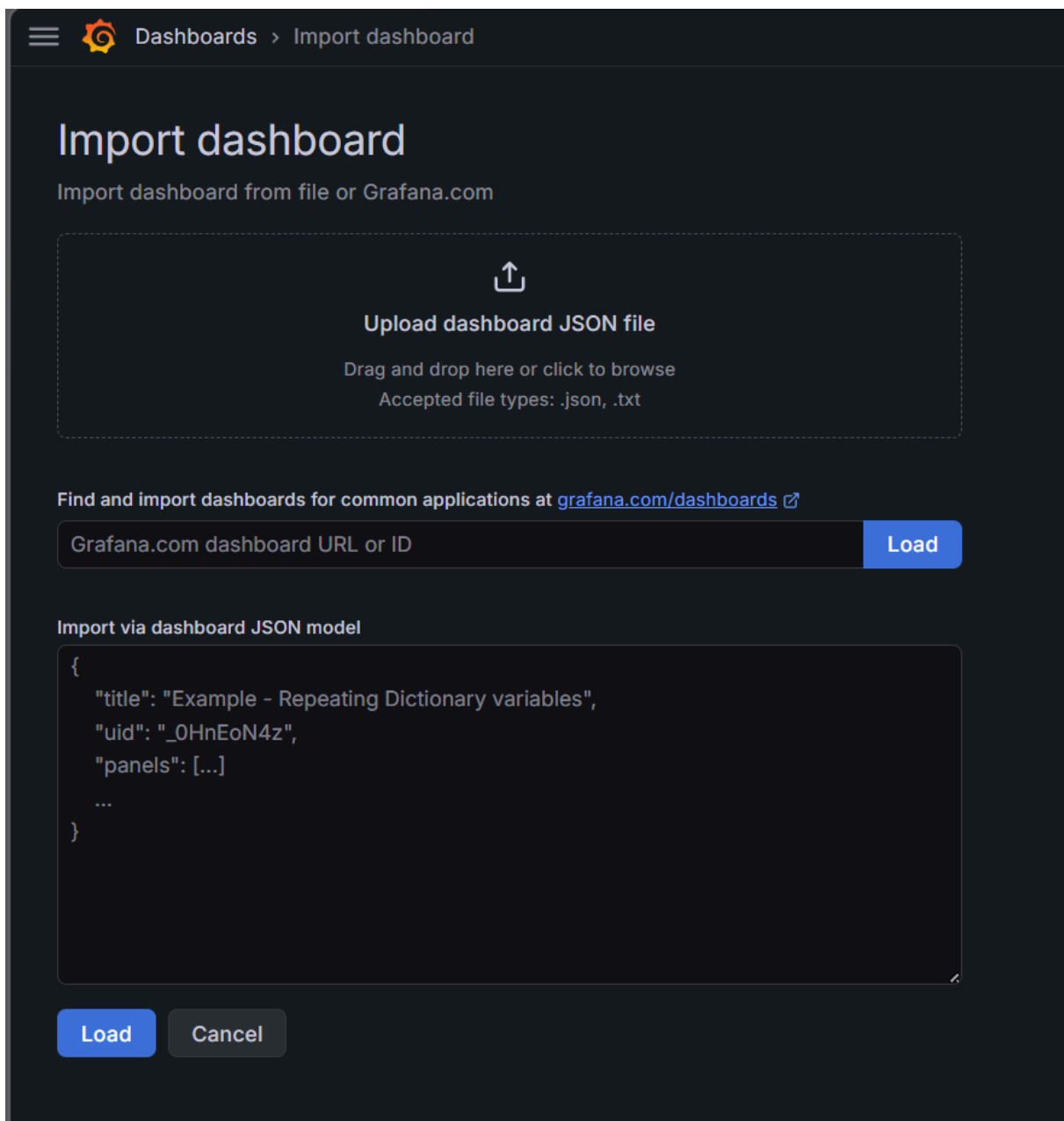
Link of Dashboard I used: [Node Exporter Full | Grafana Labs](https://grafana.com/grafana/dashboards/1860-node-exporter-full/)

**Node Exporter Full | Grafana Labs**
<https://grafana.com/grafana/dashboards/1860-node-exporter-full/>

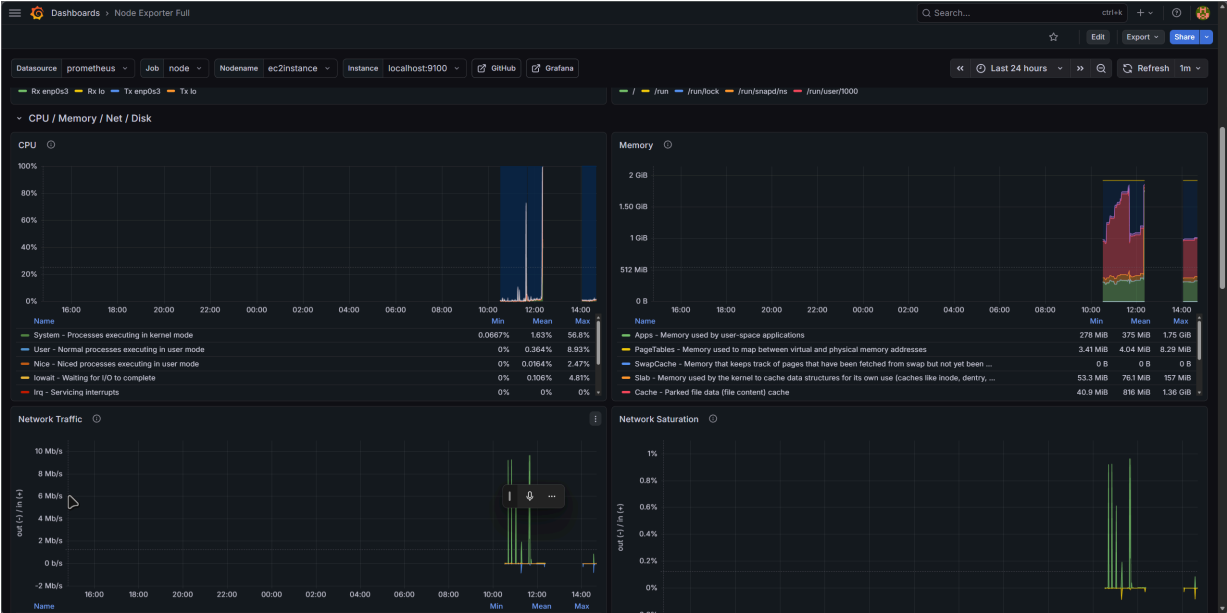
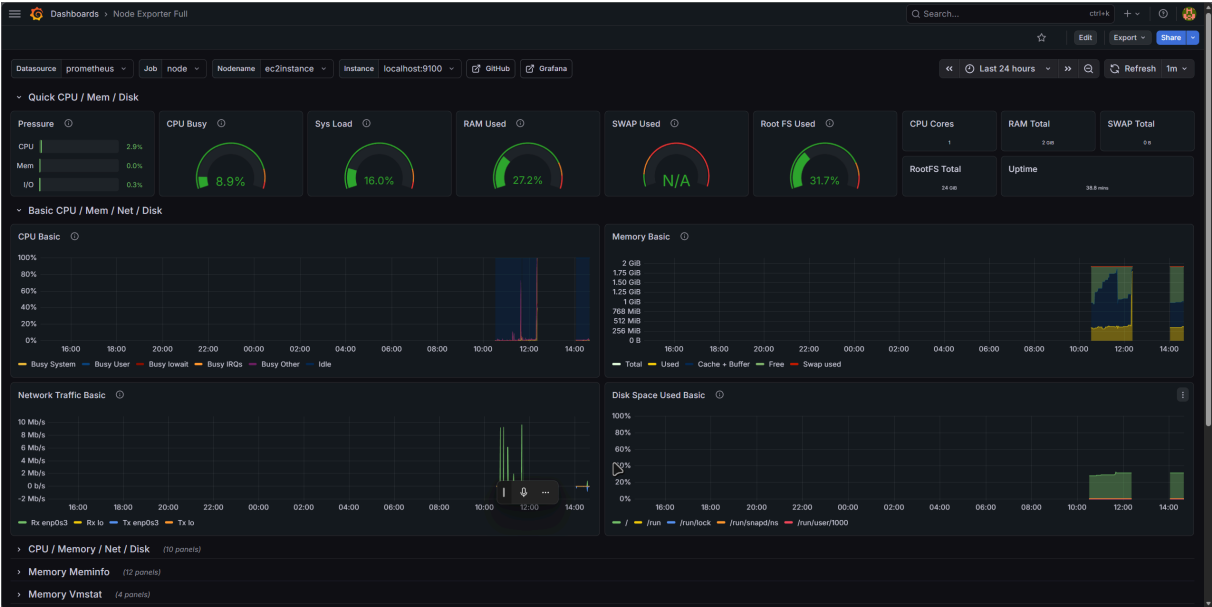


<https://grafana.com/grafana/dashboards/1860-node-exporter-full/>

- Download Lastet version of Dashboard file
- Goto Dashboard and Import
-



- Import Dashboard
- After Import Click on the Dashboard
-



Configure AWS Cloud watch and Auto scaling

- Configuring AWS Cloud watch push

```
</> Shell
1 export AWS_ACCESS_KEY_ID="AKIAZI2LFDPWIMTU22KY"
2 export AWS_SECRET_ACCESS_KEY="D2tQ/.....DCFgDojL3I2XJZCItpo0wuvb1Au"
3 export AWS_DEFAULT_REGION="ap-south-2"
```

- Check AWS Credential Working on Not.
- `aws sts get-caller-identity`
-



I tried multiple ways but Manual Script worked for me. I was not able to push logs using AWS Agent. there was some error.

- `sudo apt update``
- `sudo apt install -y python3 python3-pip unzip curl`
- `pip3 install boto3 requests`
- Create a new file
- `sudo nano /opt/prom_to_cw.py`
-

</> Python

```

1 import boto3
2 import requests
3 import time
4 import socket
5 import logging
6
7 logging.basicConfig(
8     level=logging.INFO,
9     format='%(asctime)s %(levelname)s %(message)s'
10 )
11 log = logging.getLogger(__name__)
12
13
14 PROMETHEUS_URL = "http://localhost:9090"
15 REGION = "ap-south-2"
16 NAMESPACE = "Ubuntu/Prometheus-Local-VM"
17 INTERVAL = 60
18 HOST = socket.gethostname()
19
20 METRICS = [
21     {
22         "name": "CPUUsagePercent",
23         "unit": "Percent",
24         "query": "100 - (avg(rate(node_cpu_seconds_total{mode='idle'}[2m])) * 100)"
25     },
26     {
27         "name": "MemoryUsagePercent",
28         "unit": "Percent",
29         "query": "(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100"
30     },
31     {
32         "name": "DiskUsagePercent",
33         "unit": "Percent",
34         "query": "(node_filesystem_size_bytes{mountpoint='/',fstype!='tmpfs'} -
node_filesystem_free_bytes{mountpoint='/',fstype!='tmpfs'}) /
node_filesystem_size_bytes{mountpoint='/',fstype!='tmpfs'} * 100"
35     },
36     {
37         "name": "LoadAverage1m",
38         "unit": "Count",
39         "query": "node_load1"
40     },
41     {
42         "name": "NetworkRxBytesPerSec",
43         "unit": "Bytes/Second",
44         "query": "rate(node_network_receive_bytes_total{device!='lo'}[2m])"
45     },
46     {
47         "name": "NetworkTxBytesPerSec",
48         "unit": "Bytes/Second",
49         "query": "rate(node_network_transmit_bytes_total{device!='lo'}[2m])"
50     },

```

```

51 ]
52
53 cw = boto3.client("cloudwatch", region_name=REGION)
54
55
56 def query_prometheus(promql):
57     """Query Prometheus instant query API, return float value or None."""
58     try:
59         r = requests.get(
60             f"{PROMETHEUS_URL}/api/v1/query",
61             params={"query": promql},
62             timeout=10
63         )
64         r.raise_for_status()
65         results = r.json().get("data", {}).get("result", [])
66         if not results:
67             log.warning(f"No data for query: {promql[:60]}")
68             return None
69         # Sum all results (handles multi-device metrics like network)
70         total = sum(float(r["value"][1]) for r in results)
71         return total
72     except requests.exceptions.ConnectionError:
73         log.error("Cannot connect to Prometheus – is it running on :9090?")
74     except Exception as e:
75         log.error(f"Query error: {e}")
76     return None
77
78
79 def push_metric(name, value, unit):
80     """Push a single metric to CloudWatch."""
81     try:
82         cw.put_metric_data(
83             Namespace=NAMESPACE,
84             MetricData=[{
85                 "MetricName": name,
86                 "Value": value,
87                 "Unit": unit,
88                 "Dimensions": [
89                     {"Name": "Host", "Value": HOST},
90                     {"Name": "Source", "Value": "Prometheus"}
91                 ]
92             }]
93         )
94         log.info(f" OK {name:<30} = {value:.4f} {unit}")
95     except cw.exceptions.ClientError as e:
96         log.error(f" FAIL {name}: {e.response['Error']['Message']}")
97     except Exception as e:
98         log.error(f" FAIL {name}: {e}")
99
100
101 def main():
102     log.info(f"Prometheus → CloudWatch pusher started")
103     log.info(f" Host: {HOST}")
104     log.info(f" Namespace: {NAMESPACE}")
105     log.info(f" Region: {REGION}")
106     log.info(f" Interval: {INTERVAL}s")
107     log.info(f" Metrics: {len(METRICS)}")
108
109     while True:
110         log.info(f"— pushing batch —")
111         for m in METRICS:
112             val = query_prometheus(m["query"])
113             if val is not None:
114                 push_metric(m["name"], val, m["unit"])
115         log.info(f"— done, sleeping {INTERVAL}s —")
116         time.sleep(INTERVAL)
117
118
119 if __name__ == "__main__":
120     main()

```

```
ashutosh@ec2instance ~$ nano /opt/prom_to_cw.py
GNU nano 6.2 /opt/prom_to_cw.py

    "unit": "Percent",
    "query": "100 - (avg(rate(node_cpu_seconds_total{node='idle'}[2m])) * 100)"
  },
  {
    "name": "MemoryUsagePercent",
    "unit": "Percent",
    "query": "(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100"
  },
  {
    "name": "DiskUsagePercent",
    "unit": "Percent",
    "query": "(node_filesystem_size_bytes{mountpoint='/',fstype='tmpfs'} - node_filesystem_free_bytes{mountpoint='/',fstype='tmpfs'}) / node_filesystem_size_bytes{mountpoint='/',fstype='tmpfs'} * 100"
  },
  {
    "name": "LoadAverage1m",
    "unit": "Count",
    "query": "node_load1"
  },
  {
    "name": "NetworkRxBytesPerSec",
    "unit": "Bytes/Second",
    "query": "rate(node_network_receive_bytes_total{device='lo'}[2m])"
  },
  {
    "name": "NetworkTxBytesPerSec",
    "unit": "Bytes/Second",
    "query": "rate(node_network_transmit_bytes_total{device='lo'}[2m])"
  },
}

cw = boto3.client('cloudwatch', region_name=REGION)

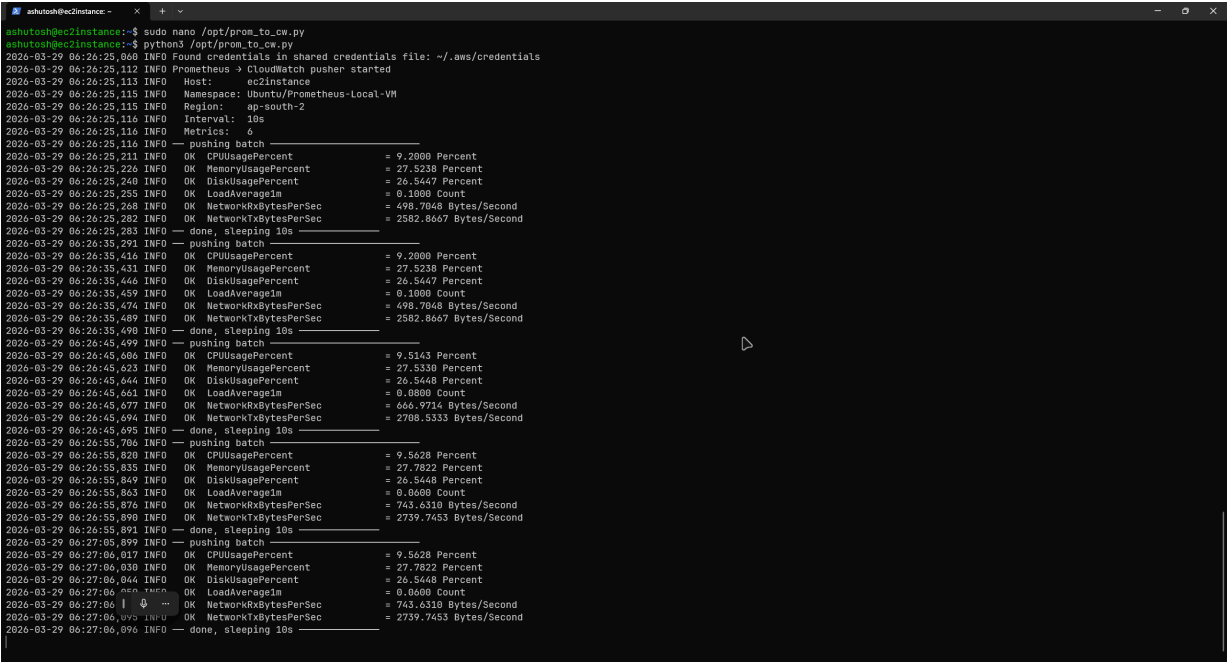
def query_prometheus(promql):
    """Query Prometheus instant query API, return float value or None."""
    try:
        r = requests.get(
            f="{PROMTHEUS_URL}/api/v1/query",
            params={'query': promql},
            timeout=10
        )
        r.raise_for_status()
        results = r.json().get("data", {}).get("result", [])
        if not results:
            log.warning("No data for query: {promql:60}")
            return None
    except:
        return None

# Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket Previous Back Prev Word
# Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was Next Forward Next Word
```

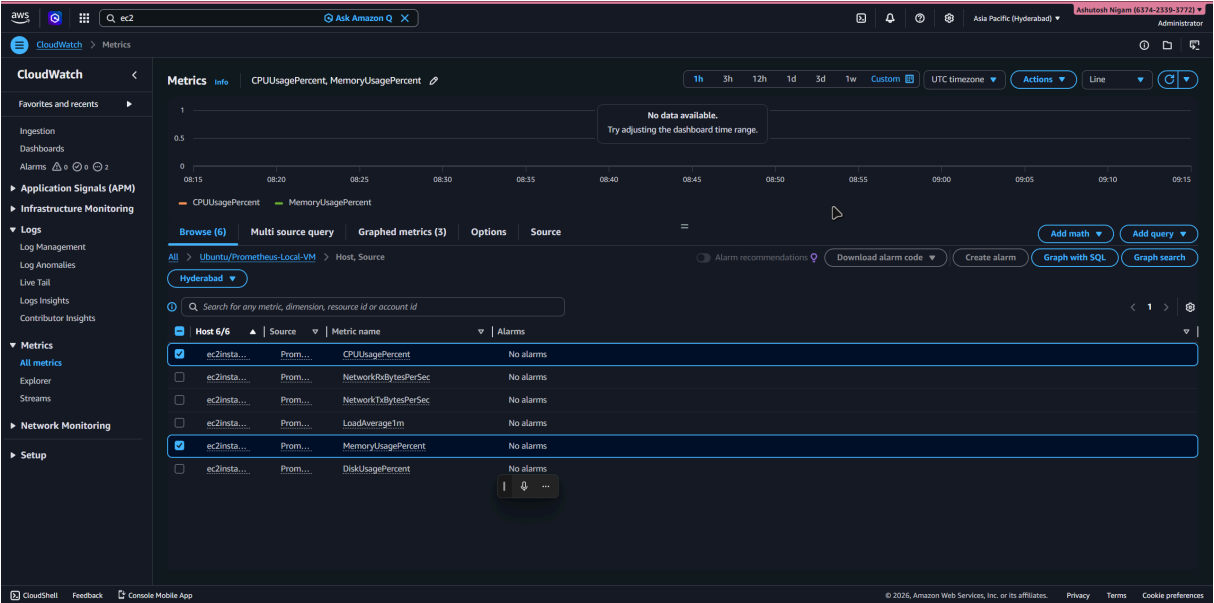
- Change owner of this file `sudo chmod +x /opt/prom_to_cw.py`

- Test: `python3 /opt/prom_to_cw.py`

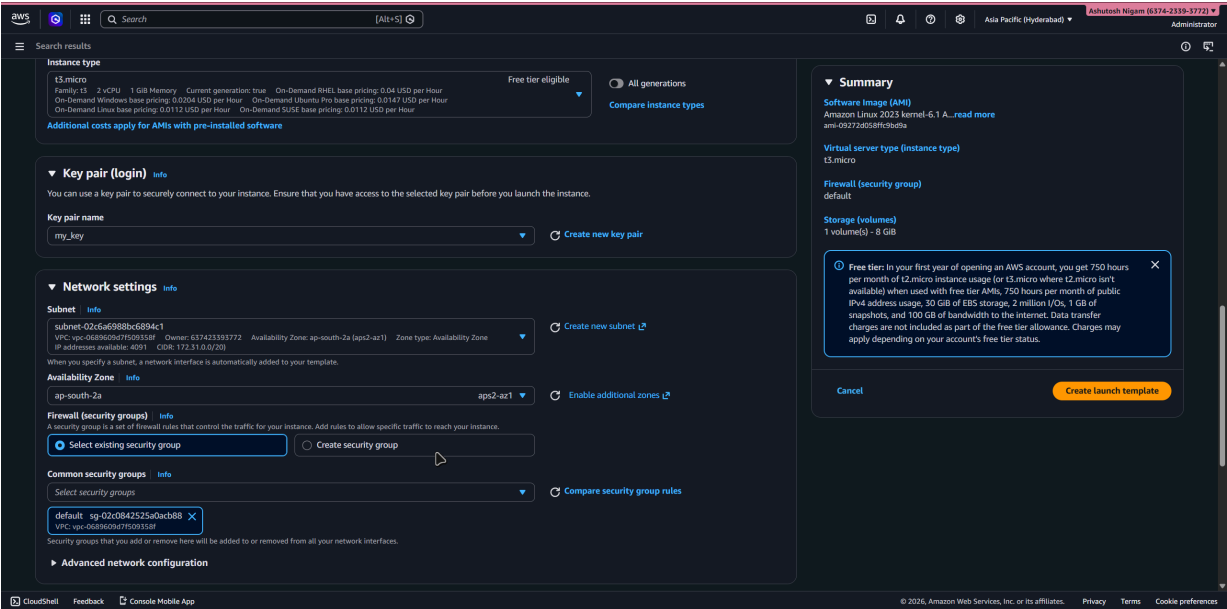
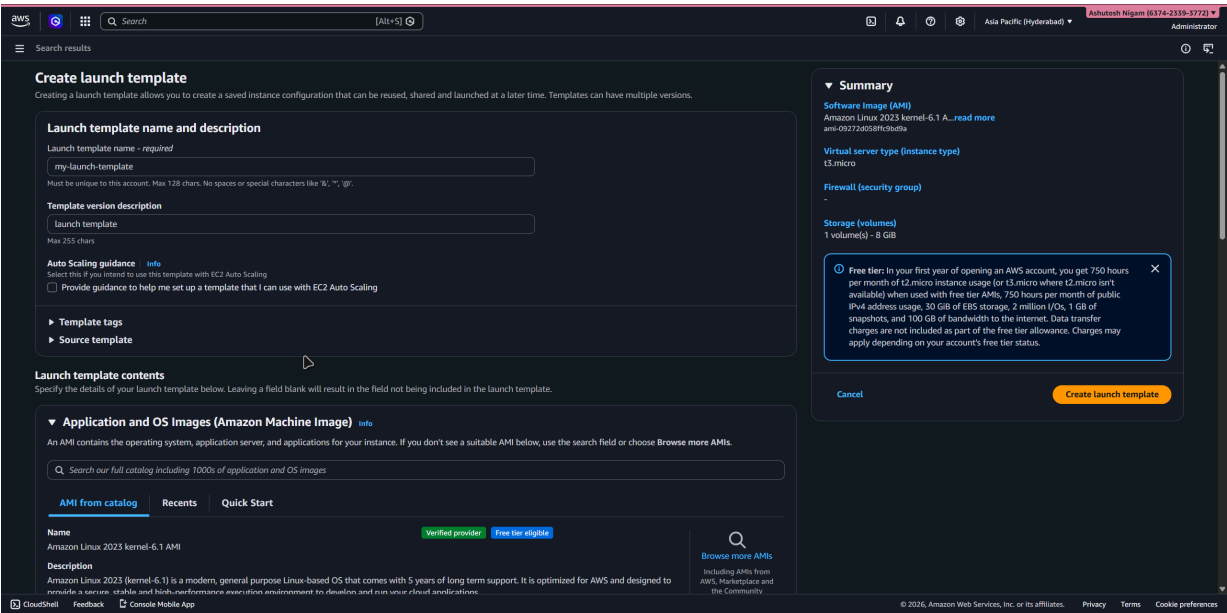
```
ashutosh@ec2instance ~$ nano /opt/prom_to_cw.py
ashutosh@ec2instance ~$ sudo nano /opt/prom_to_cw.py
ashutosh@ec2instance ~$ python3 /opt/prom_to_cw.py
2026-03-29 06:17:25,625 INFO Found credentials in shared credentials file: ~/.aws/credentials
2026-03-29 06:17:25,682 INFO Prometheus -> CloudWatch pusher started
2026-03-29 06:17:25,684 INFO Host: ec2instance
2026-03-29 06:17:25,685 INFO Namespace: Ubuntu/Prometheus-Local-VM
2026-03-29 06:17:25,686 INFO Region: ap-south-2
2026-03-29 06:17:25,686 INFO Interval: 60s
2026-03-29 06:17:25,686 INFO Metrics: 6
2026-03-29 06:17:25,686 INFO - pushing batch -
2026-03-29 06:17:25,816 INFO OK CPUUsagePercent = 9.7463 Percent
2026-03-29 06:17:25,831 INFO OK MemoryUsagePercent = 26.9880 Percent
2026-03-29 06:17:25,844 INFO OK DiskUsagePercent = 26.5429 Percent
2026-03-29 06:17:25,857 INFO OK LoadAverage1m = 0.2100 Count
2026-03-29 06:17:25,872 INFO OK NetworkRxBytesPerSec = 888.6000 Bytes/Second
2026-03-29 06:17:25,886 INFO OK NetworkTxBytesPerSec = 2763.0857 Bytes/Second
2026-03-29 06:17:25,888 INFO - done, sleeping 60s -
```

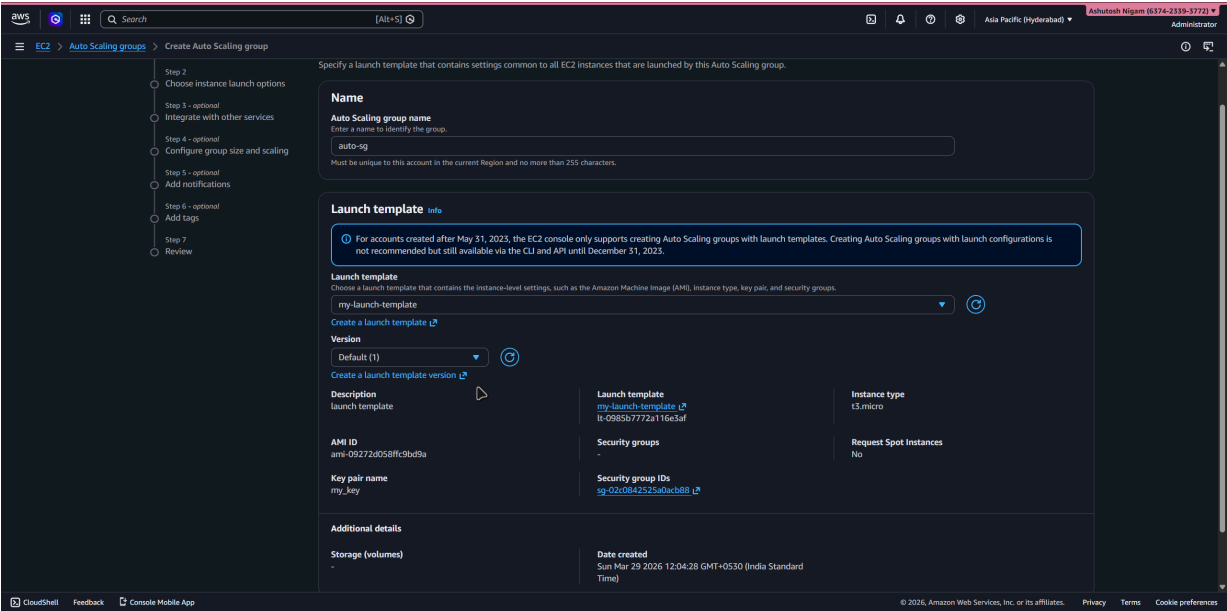


- Cloud watch Metrics
-

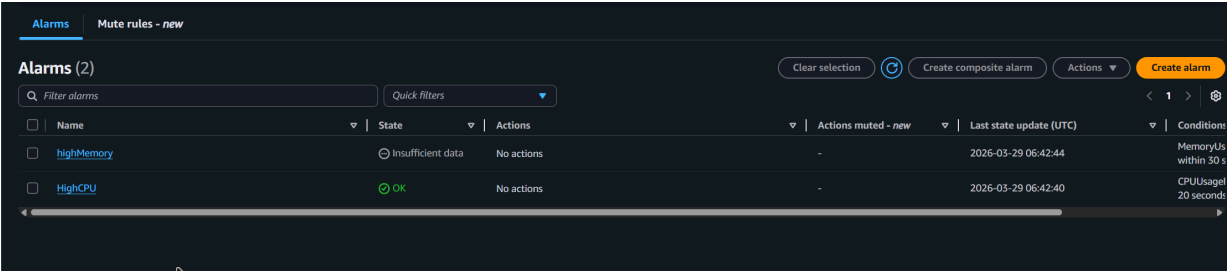
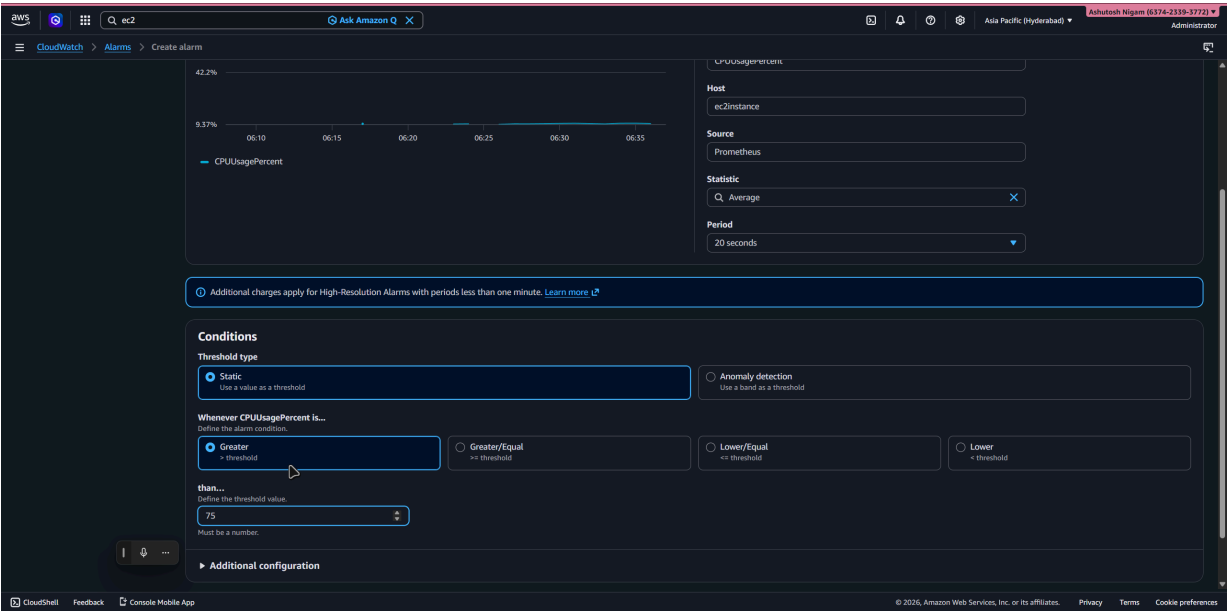


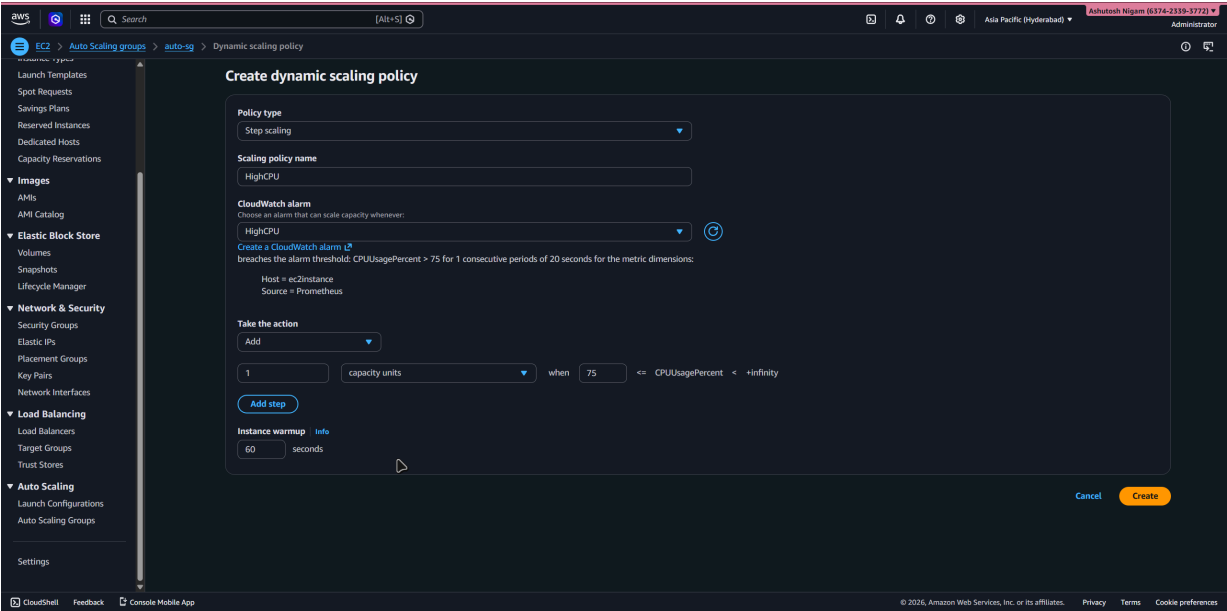
Create AWS Launch Template:



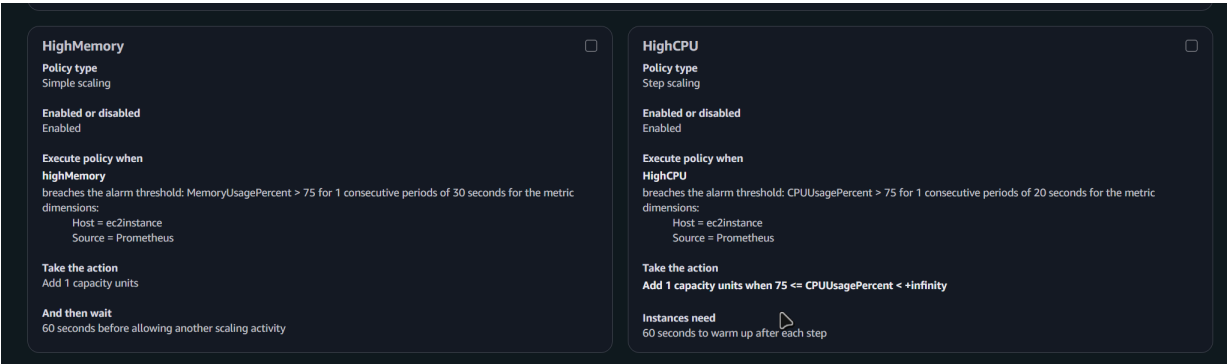


- Setting CloudWatch Alarm



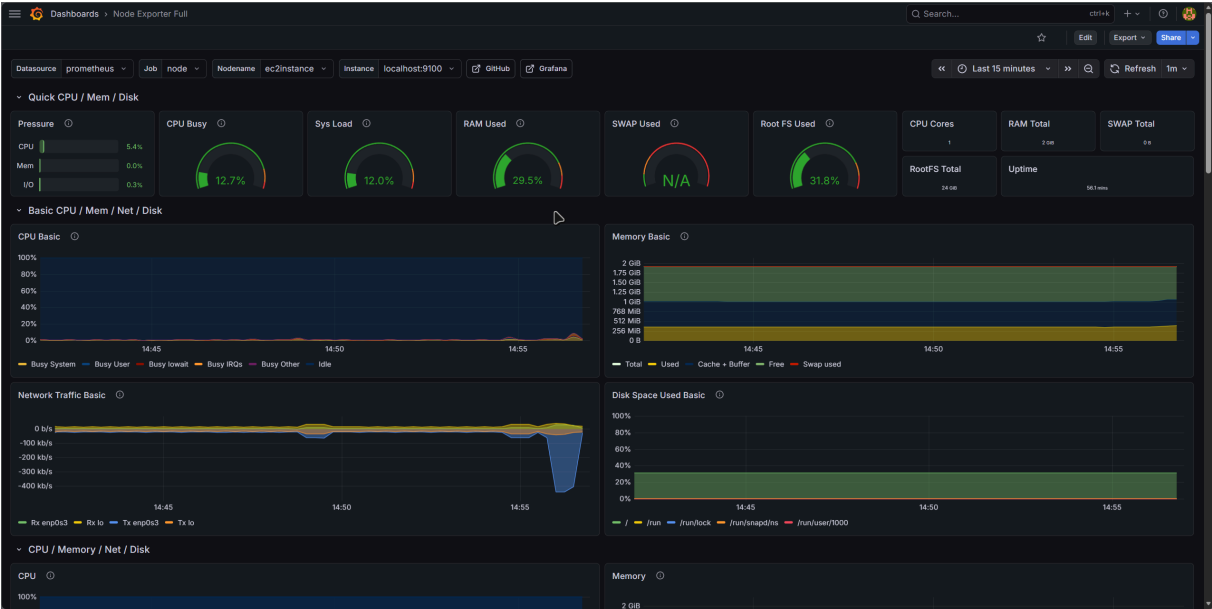


Adding Both Metrics into Auto Scaling Group



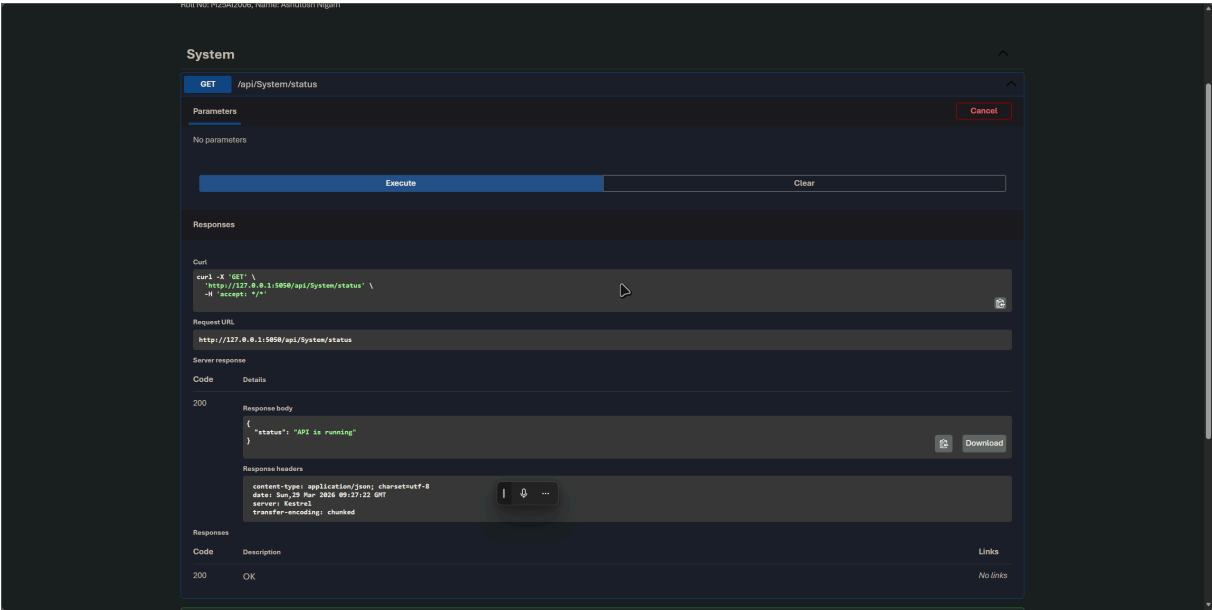
Testing Application

- Grafana
-



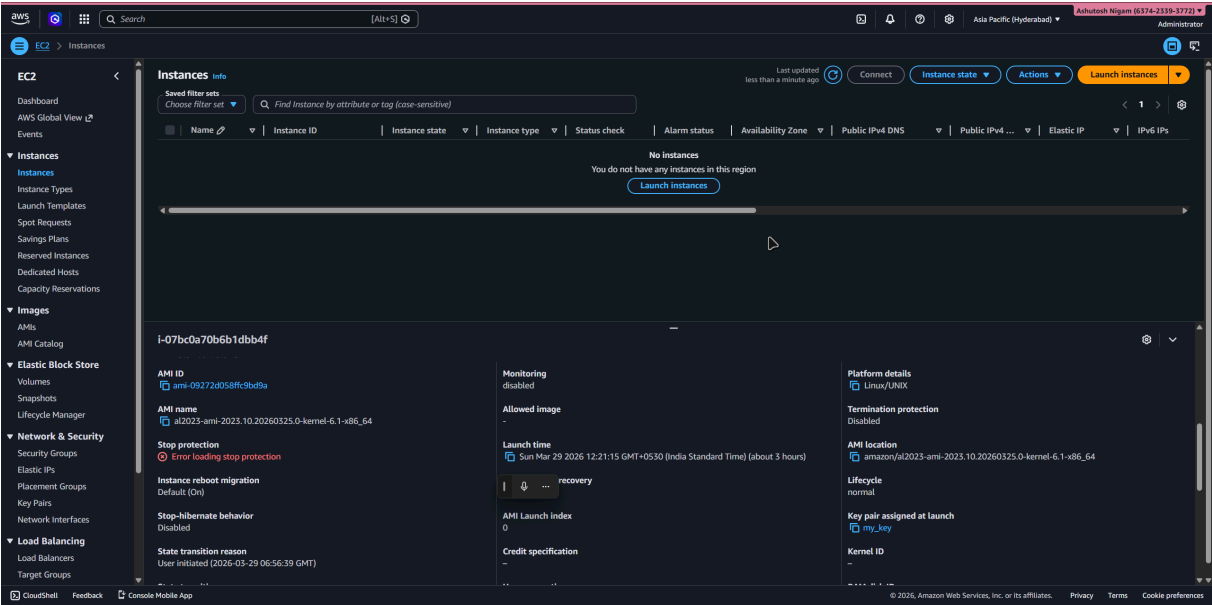
- Service- Running

•



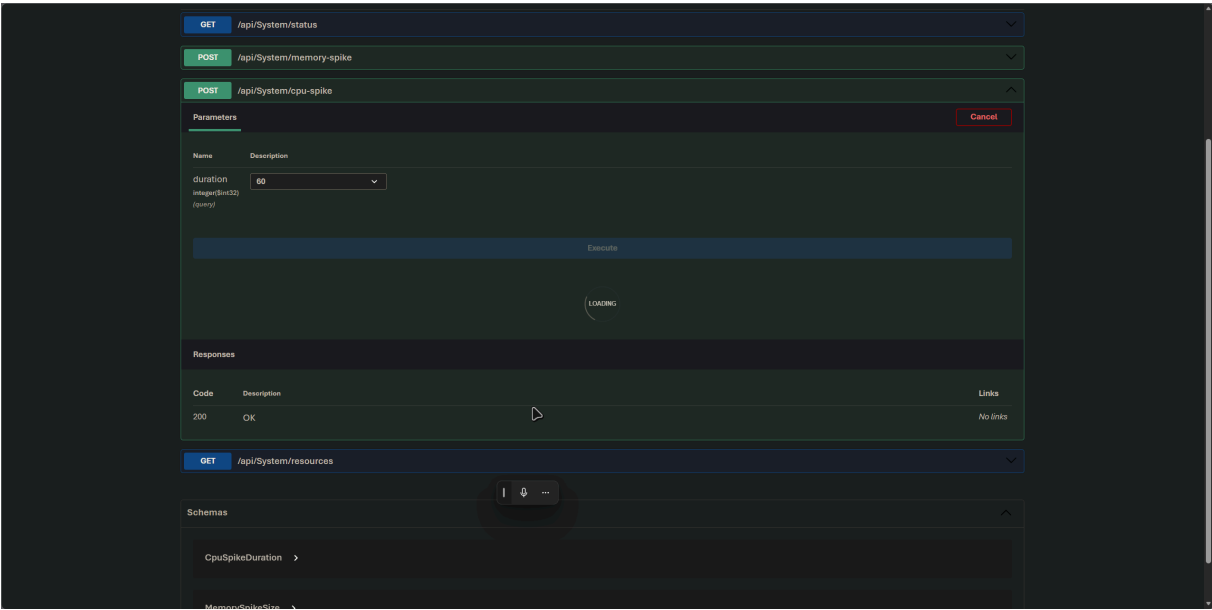
- AWS Instances - 0

•



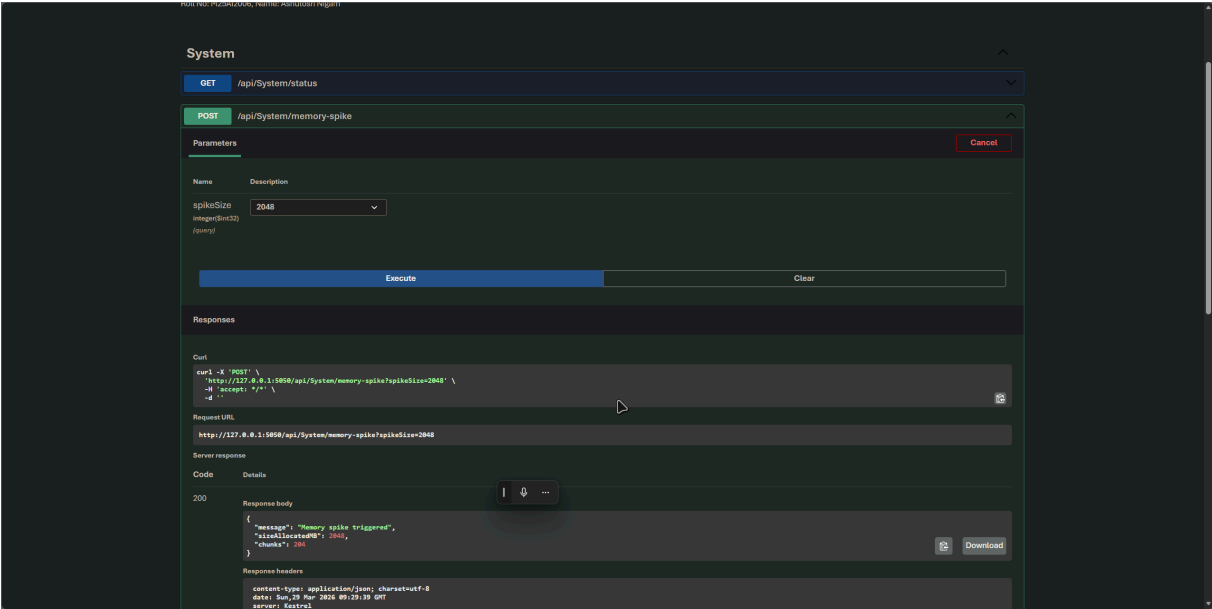
- CPU Spike

•

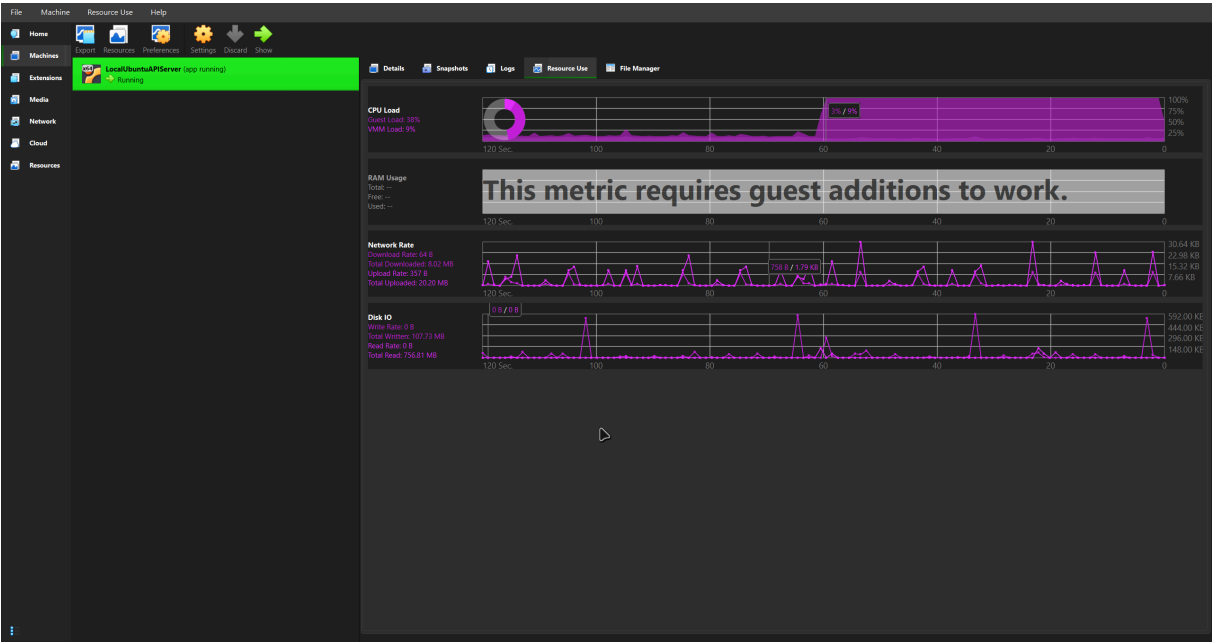


- Memory Spike

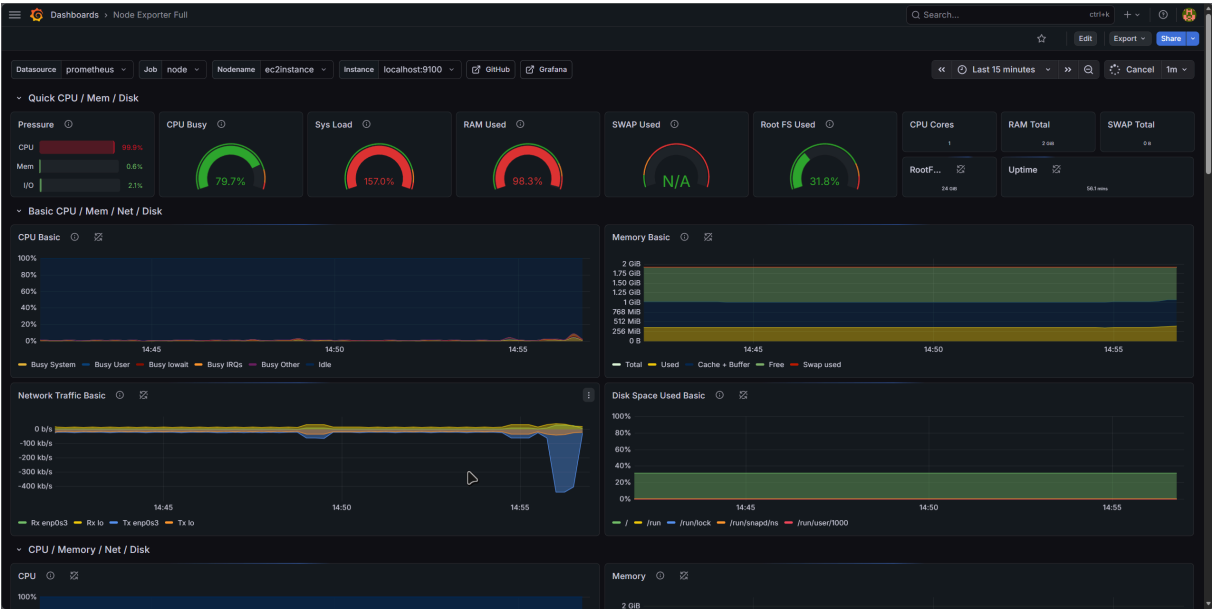
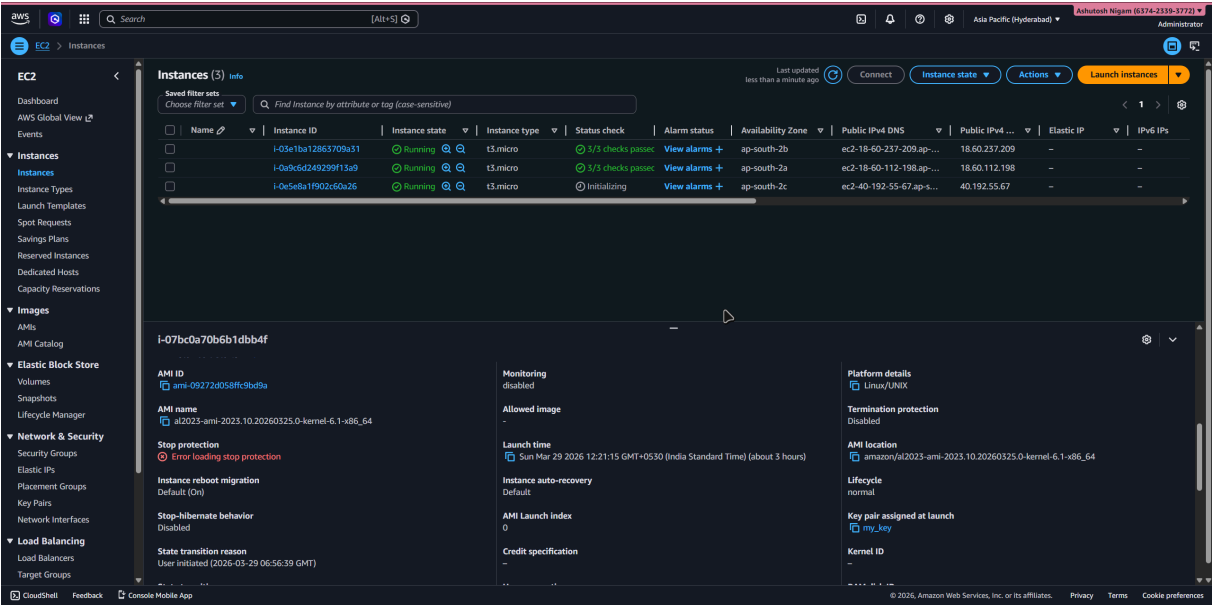
•



- CPU & Memory



- New Instances in AWS



References:

- [How to Install and Configure Prometheus a...](https://www.linode.com/docs/guides/how-to-install-prometheus-and-grafana-on-ubuntu/)
- [Install Grafana | Grafana documentation](https://grafana.com/docs/grafana/latest/setup-grafana/installation/)
- [Node Exporter Full | Grafana Labs](https://grafana.com/grafana/dashboards/1860-node-exporter-full/)
- [Prometheus vs Grafana: Knowing the Differ...](https://www.opsramp.com/guides/prometheus-monitoring/prometheus-vs-grafana/)

